Introduction to Linear Programming

BUS 104: Decision Analysis and Management Science

Danko Turcic

October 14, 2026

UC RIVERSIDE | UNIVERSITY OF CALIFORNIA School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Agenda: Linear Programming Applications

Welcome! In this presentation, we will explore the power of Linear Programming (LP) through a series of foundational business and operational problems.

We will cover how to formulate and solve:

1. **Product Mix Problem:** Maximizing manufacturing profit (Par, Inc.)
2. **Blending Problems (I & II):** Mixing ingredients to meet specifications at lowest cost (e.g., Coffee Blending)
3. **Work Scheduling Problem:** Assigning labor shifts to meet demand
4. **Transportation Problem:** Minimizing shipping costs from origins to destinations
5. **Assignment Problem:** Optimal 1-to-1 resource allocation (e.g., Project Staffing)
6. **Shortest Path Problem:** Finding the most efficient route through a network

Our Approach

For each problem category, we will follow a rigorous 3-step methodology:

- **Step 1: The Business Problem.** Understanding the scenario, the physical constraints, and the managerial objective.
- **Step 2: Mathematical Formulation.** Translating English into Algebra (defining Decision Variables, Objective Function, and Constraints).
- **Step 3: Python Implementation.** Using the `PuLP` library to computationally solve the model and extract business insights.

Let's begin with our first category: The Product Mix Problem.

Product Mix Example: Par, Inc.

Goal: Maximize weekly profit by deciding how many standard and deluxe golf bags to produce.

The Data:

Department	**Standard Bag**	**Deluxe Bag**	**Hours Available**
Cutting & Dyeing	7/10 hr	1 hr	630 hrs
Sewing	1/2 hr	5/6 hr	600 hrs
Finishing	1 hr	2/3 hr	708 hrs
Inspection & Pack.	0.1 hr	1/4 hr	135 hrs
Profit/Bag	**$10**	**$9**	

- *Assumption:* Demand is unlimited; we can sell everything we make.
- **Question:** Formulate a mathematical model to find the optimal production plan.

Par, Inc. Golf Bag Production Problem

Decision variables.

Let x_s = number of standard golf bags produced per week,
and x_d = number of deluxe golf bags produced per week.

Both $x_s, x_d \geq 0$.

Par, Inc. Golf Bag Production Problem

Decision variables.
Let x_s = number of standard golf bags produced per week,
and x_d = number of deluxe golf bags produced per week.

Both $x_s, x_d \geq 0$.

Objective function.
Maximize weekly profit:

$$\max \ Z = 10x_s + 9x_d.$$

Subject to capacity constraints:

$$\frac{7}{10}x_s + 1x_d \leq 630 \qquad \text{(Cutting \& Dyeing)}$$

Par, Inc. Golf Bag Production Problem

Decision variables.
Let x_s = number of standard golf bags produced per week,
and x_d = number of deluxe golf bags produced per week.

Both $x_s, x_d \geq 0$.

Objective function.
Maximize weekly profit:

$$\max \ Z = 10x_s + 9x_d.$$

Subject to capacity constraints:

$$\frac{7}{10}x_s + 1x_d \leq 630 \qquad \text{(Cutting \& Dyeing)}$$
$$\frac{1}{2}x_s + \frac{5}{6}x_d \leq 600 \qquad \text{(Sewing)}$$

Par, Inc. Golf Bag Production Problem

Decision variables.
Let x_s = number of standard golf bags produced per week,
and x_d = number of deluxe golf bags produced per week.

Both $x_s, x_d \geq 0$.

Objective function.
Maximize weekly profit:

$$\max \ Z = 10x_s + 9x_d.$$

Subject to capacity constraints:

$$\begin{aligned} \tfrac{7}{10}x_s + 1x_d &\leq 630 && \text{(Cutting \& Dyeing)} \\ \tfrac{1}{2}x_s + \tfrac{5}{6}x_d &\leq 600 && \text{(Sewing)} \\ 1x_s + \tfrac{2}{3}x_d &\leq 708 && \text{(Finishing)} \end{aligned}$$

Par, Inc. Golf Bag Production Problem

Decision variables.
Let x_s = number of standard golf bags produced per week,
and x_d = number of deluxe golf bags produced per week.

Both $x_s, x_d \geq 0$.

Objective function.
Maximize weekly profit:

$$\max \ Z = 10x_s + 9x_d.$$

Subject to capacity constraints:

$$\begin{aligned}
\tfrac{7}{10}x_s + 1x_d &\leq 630 && \text{(Cutting \& Dyeing)} \\
\tfrac{1}{2}x_s + \tfrac{5}{6}x_d &\leq 600 && \text{(Sewing)} \\
1x_s + \tfrac{2}{3}x_d &\leq 708 && \text{(Finishing)} \\
0.1x_s + \tfrac{1}{4}x_d &\leq 135 && \text{(Inspection \& Packaging)} \\
x_s, x_d &\geq 0.
\end{aligned}$$

Par, Inc. Golf Bag Production Solution (PuLP)

```
import pulp as pl

# Create the problem
prob = pl.LpProblem("Par_Inc_Golf_Bags", pl.LpMaximize)

# Define variables directly into the model
x_s = prob.add_variable("Standard_Bags", lowBound=0)
x_d = prob.add_variable("Deluxe_Bags", lowBound=0)

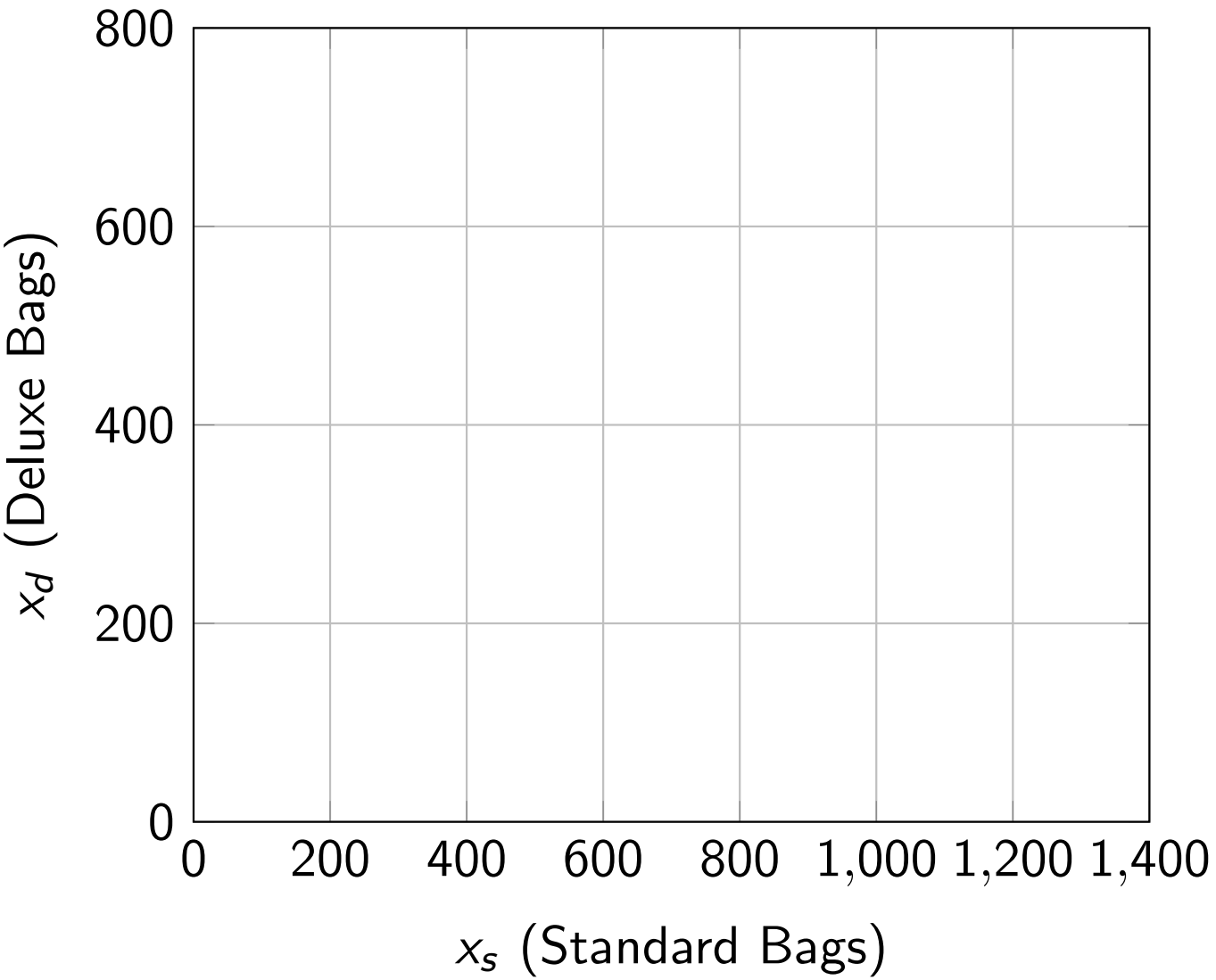
# Objective function
prob += 10 * x_s + 9 * x_d, "Maximize_Profit"

# Constraints
prob += 0.7 * x_s + 1 * x_d <= 630, "Cutting_Dyeing"
prob += 0.5 * x_s + 5/6 * x_d <= 600, "Sewing"
prob += 1 * x_s + 2/3 * x_d <= 708, "Finishing"
prob += 0.1 * x_s + 0.25 * x_d <= 135, "Inspection_Packaging"

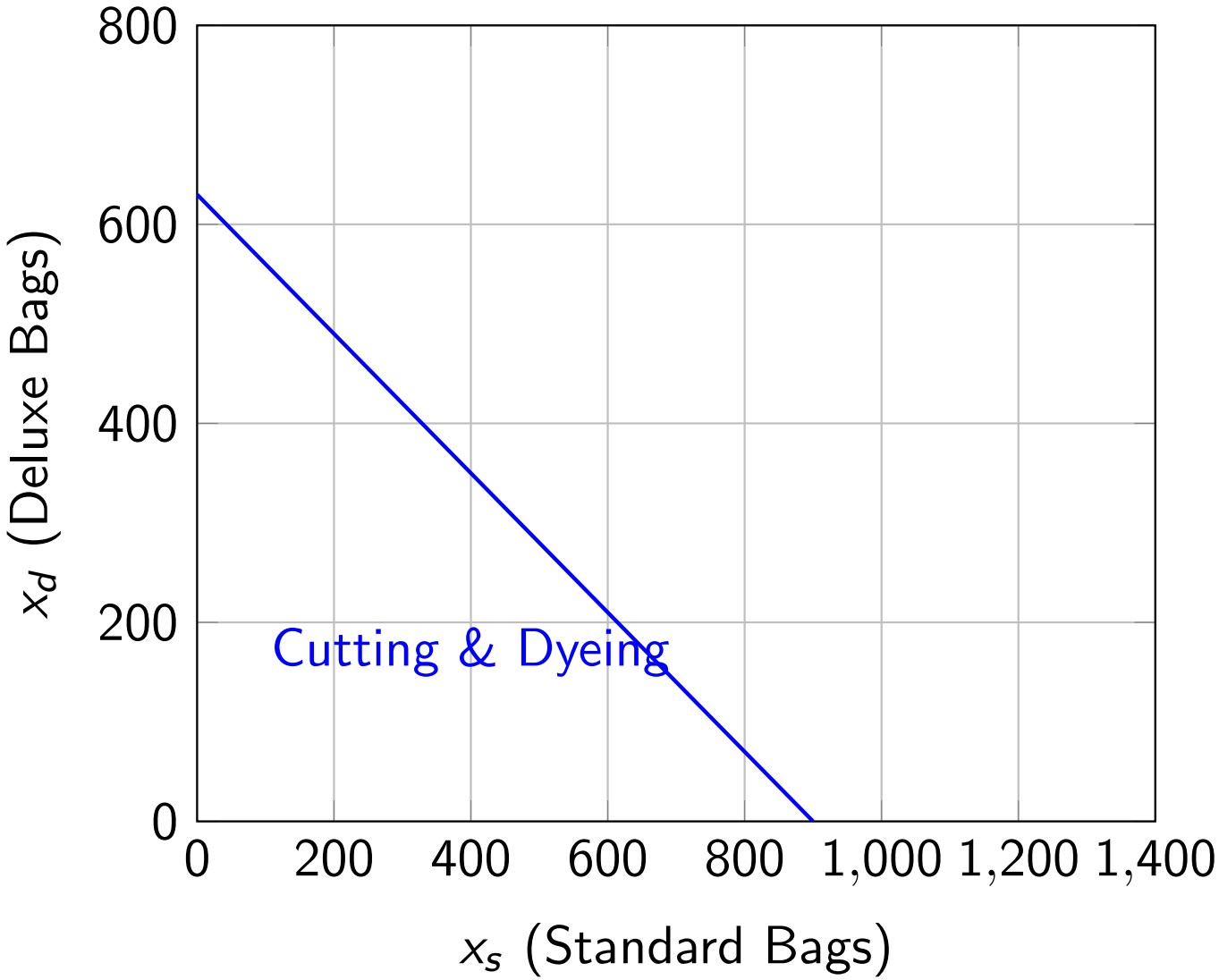
# Solve the problem
res = prob.solve()

# Print results
print("Status:", res.status.name)
print("Optimal Solution:")
print(f"Standard Bags = {x_s.varValue}")
print(f"Deluxe Bags = {x_d.varValue}")
print(f"Maximum Profit = ${pl.value(prob.objective)}")
```

Graphical Solution – Step 1: Set up the Axes



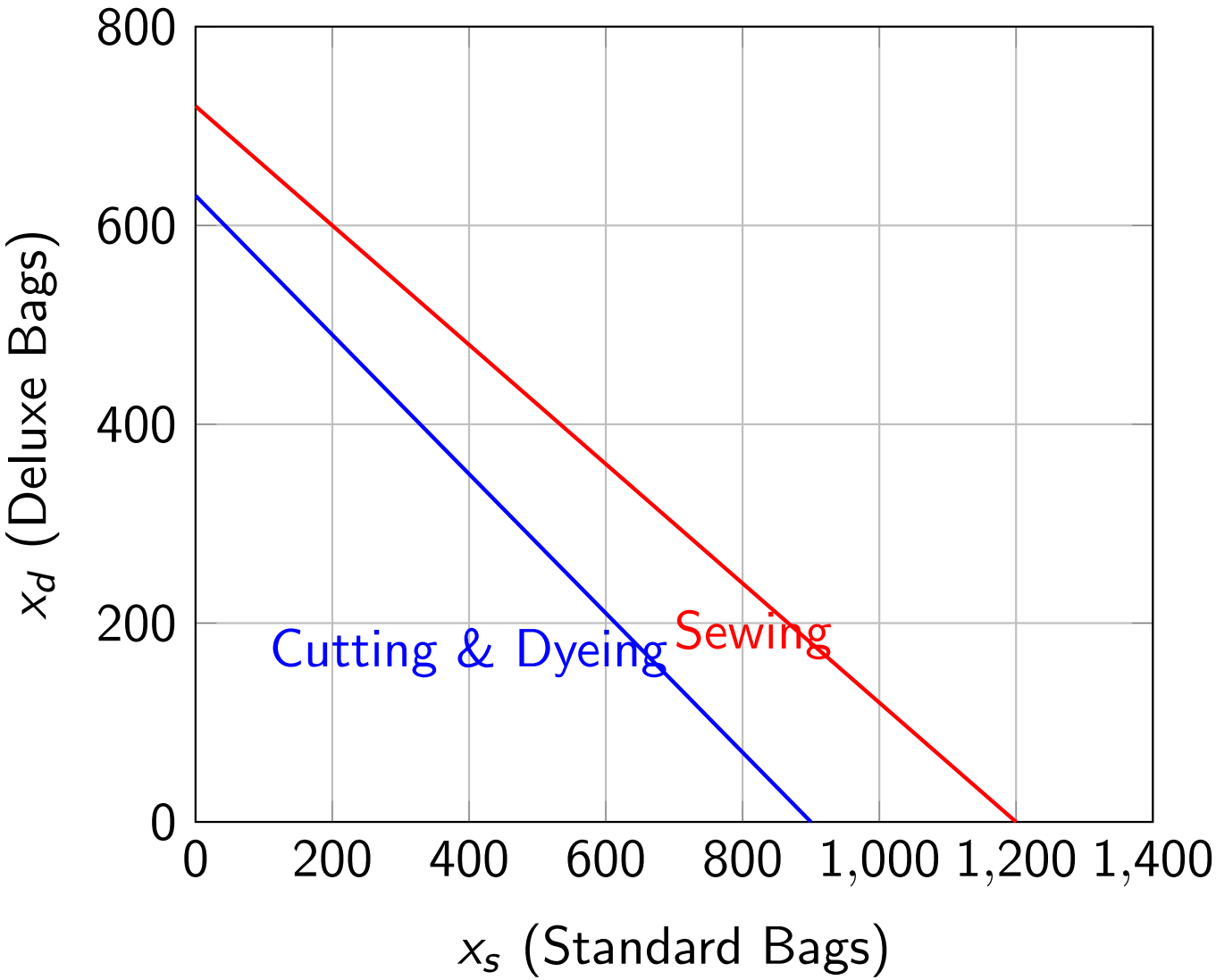
Graphical Solution – Step 2: Add C&D Constraint



$0.7x_s + x_d \leq 630$

Feasible region is below the line.

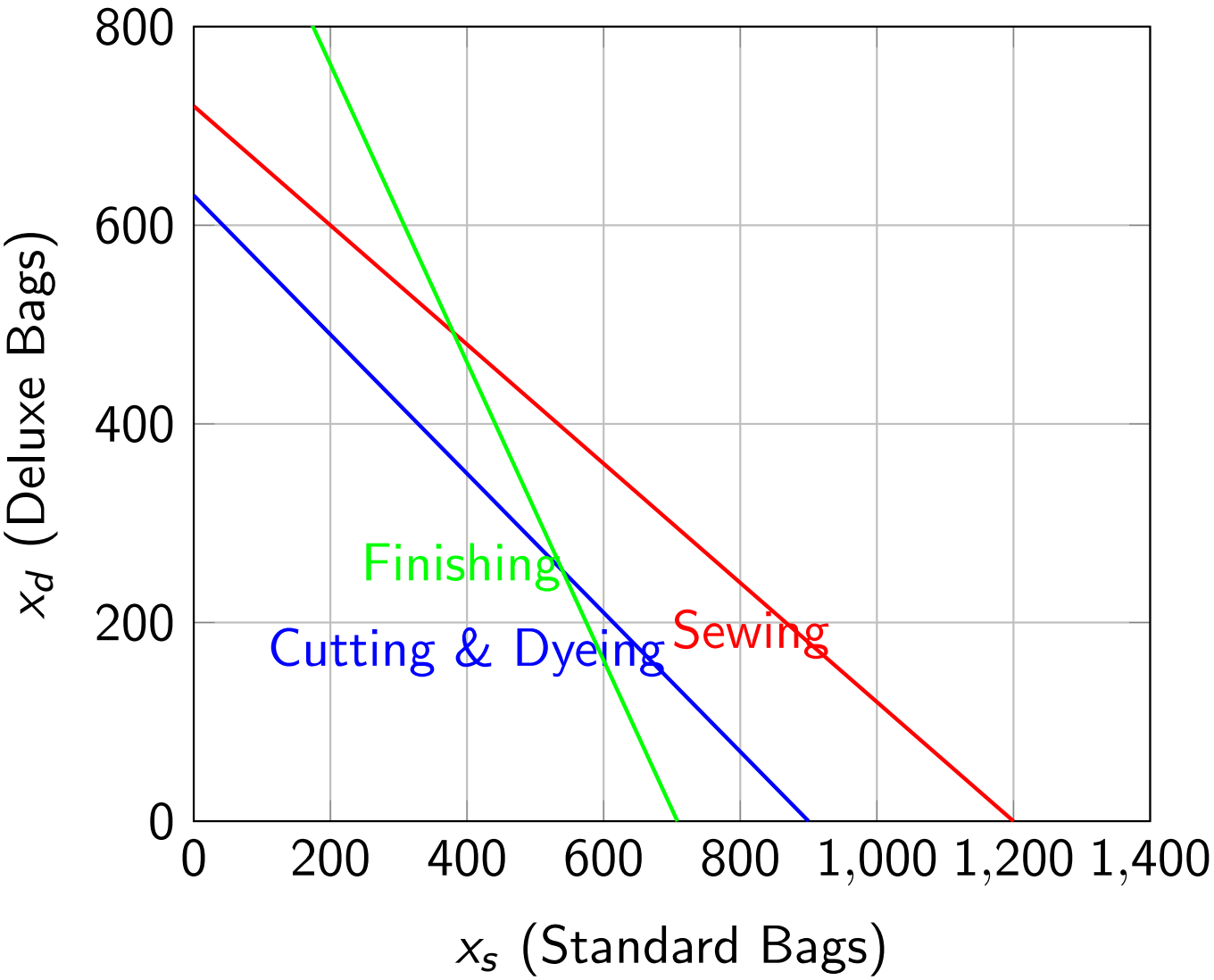
Graphical Solution – Step 3: Add Sewing Constraint



$0.5x_s + \frac{5}{6}x_d \leq 600$

Feasible region is below the line.

Graphical Solution – Step 4: Add Finishing Constraint

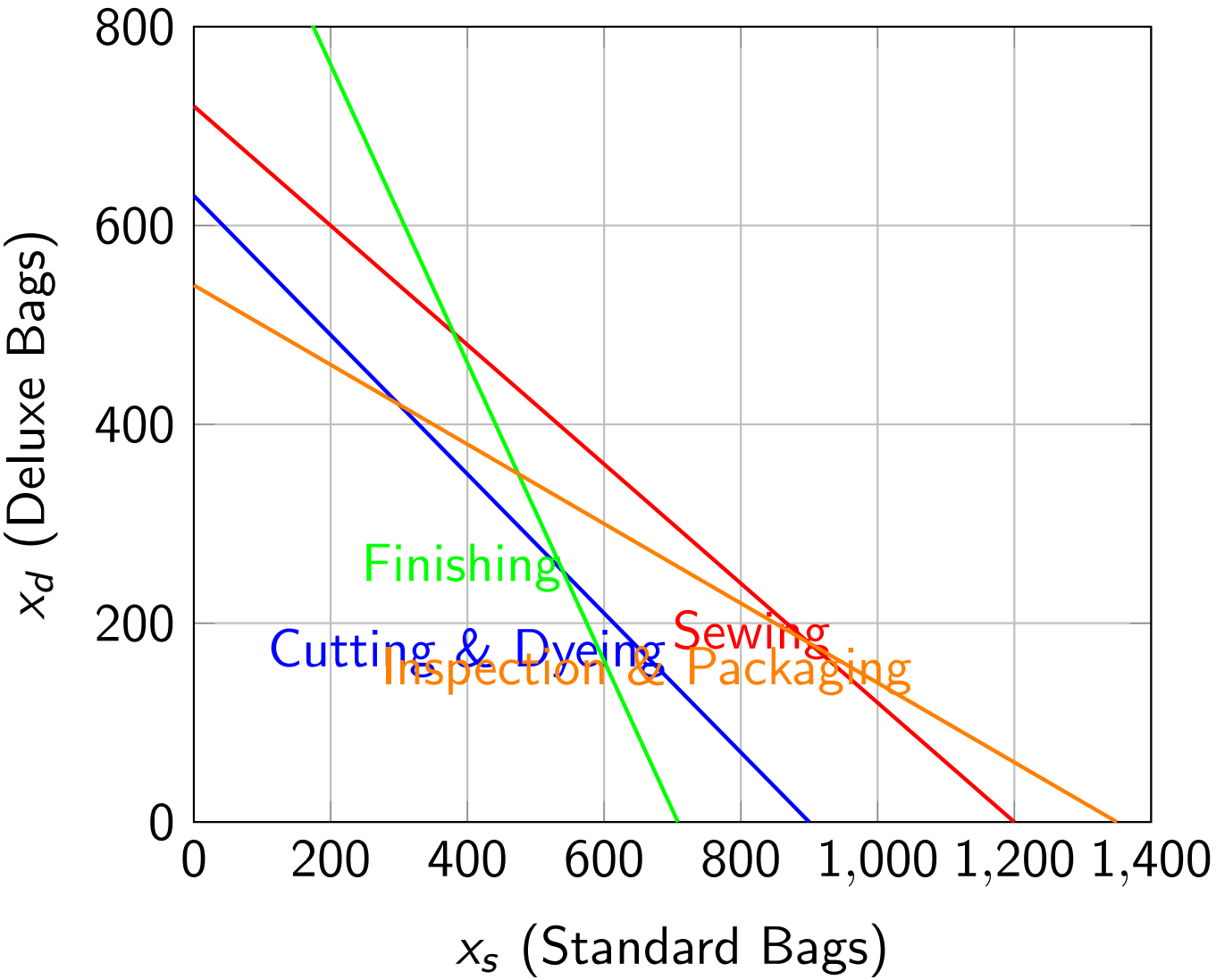


$x_s + \frac{2}{3}x_d \leq 708$

Feasible region is below the line.

Graphical Solution – Step 5: Add Inspection Constraint

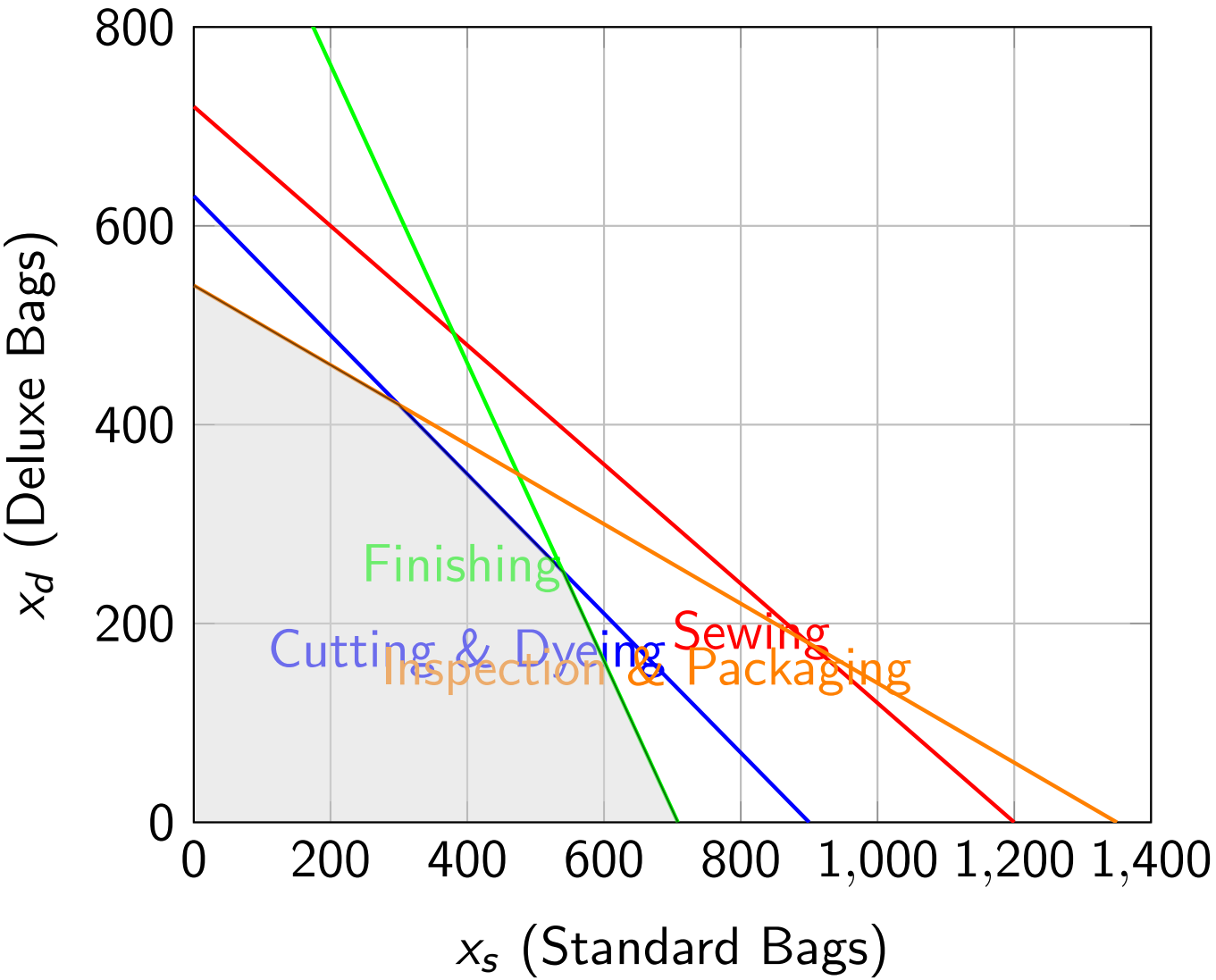
$0.1x_s + 0.25x_d \leq 135$



Feasible region is below the line.

Graphical Solution – Step 6: Feasible Region

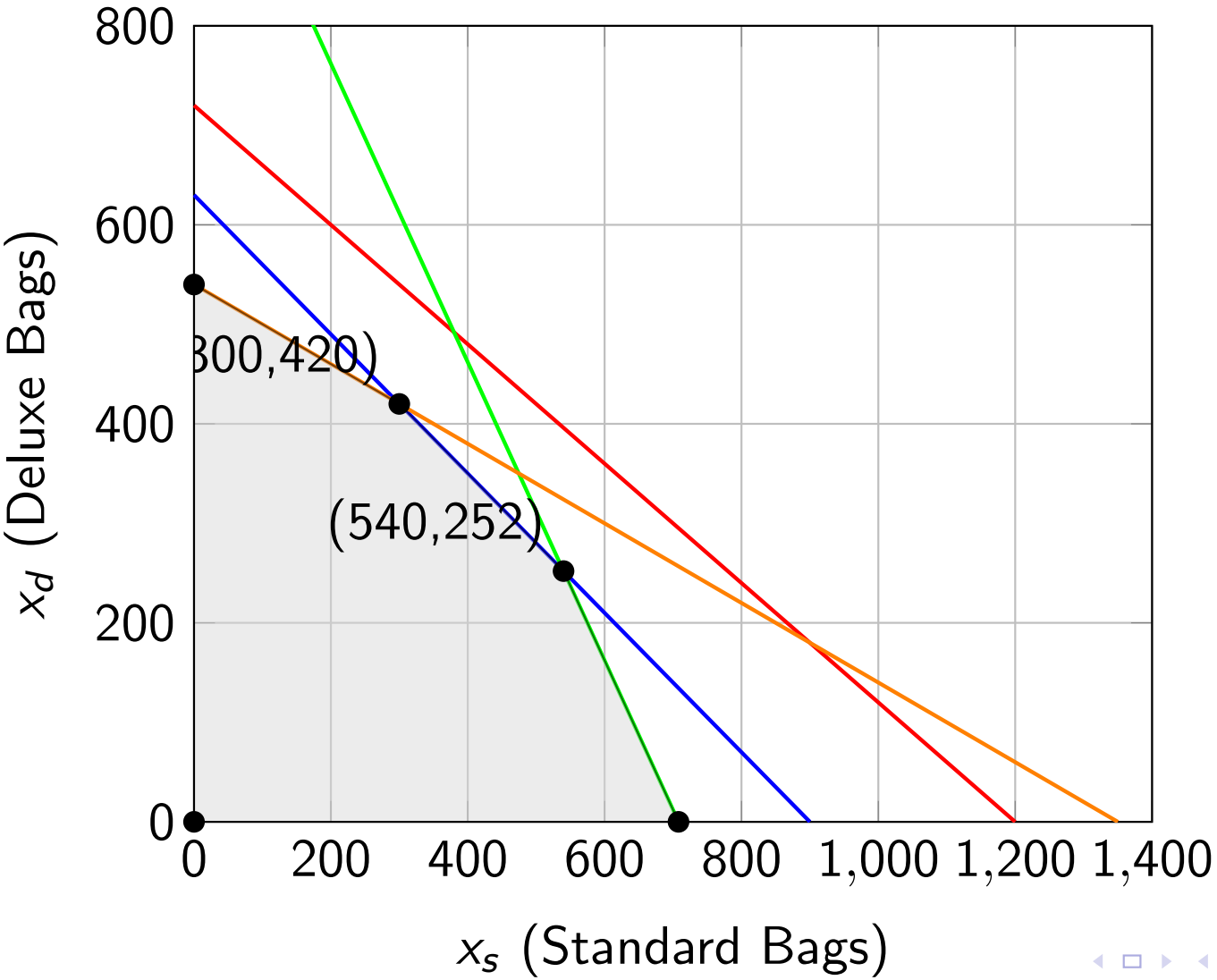
The feasible region is the area below all constraint lines and above the



axes.

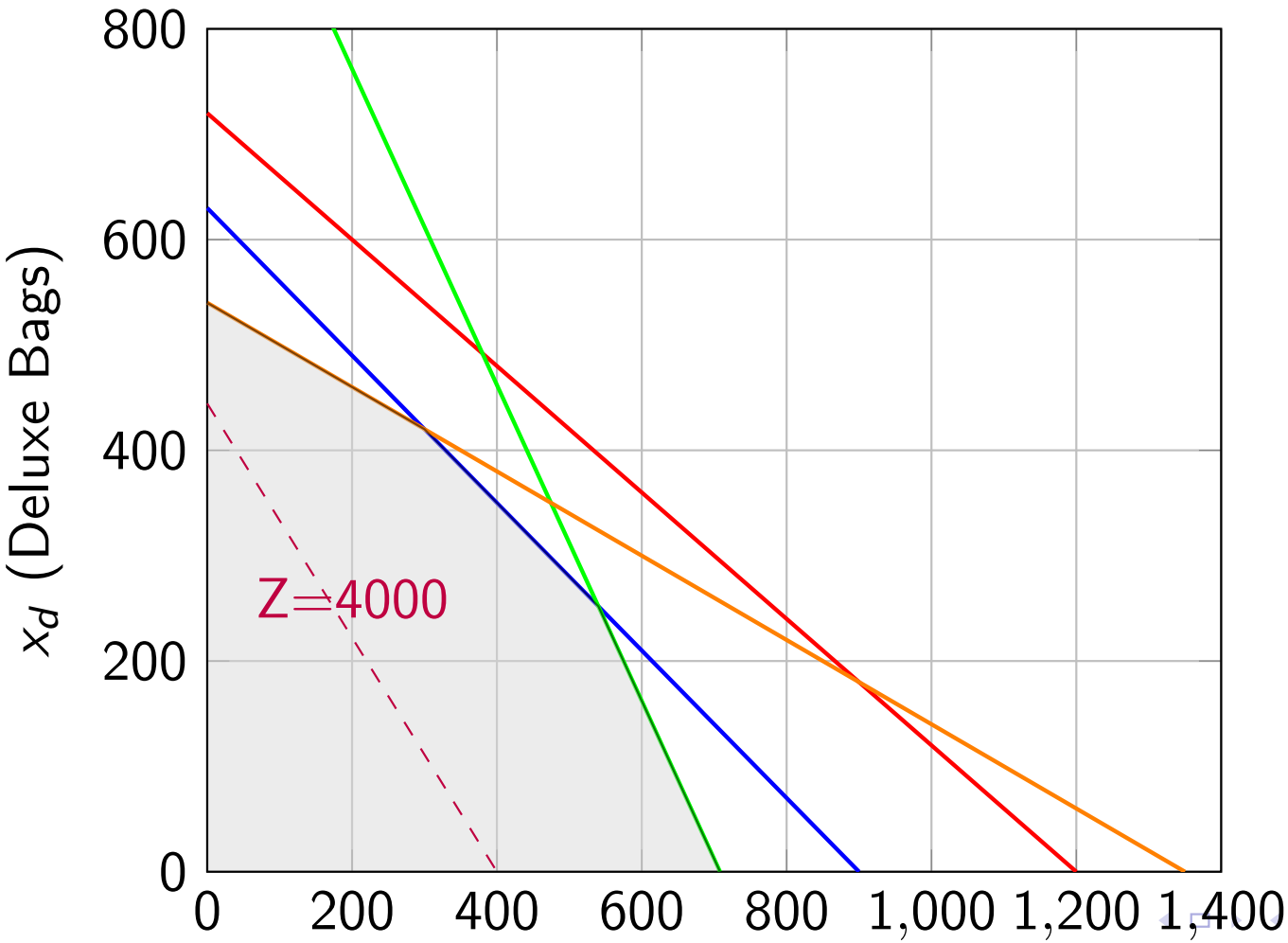
Graphical Solution – Step 7: Corner Points

Identify the corner points of the feasible region: - (0, 0) - (0, 540) - (300, 420) - (540, 252) - (708, 0)



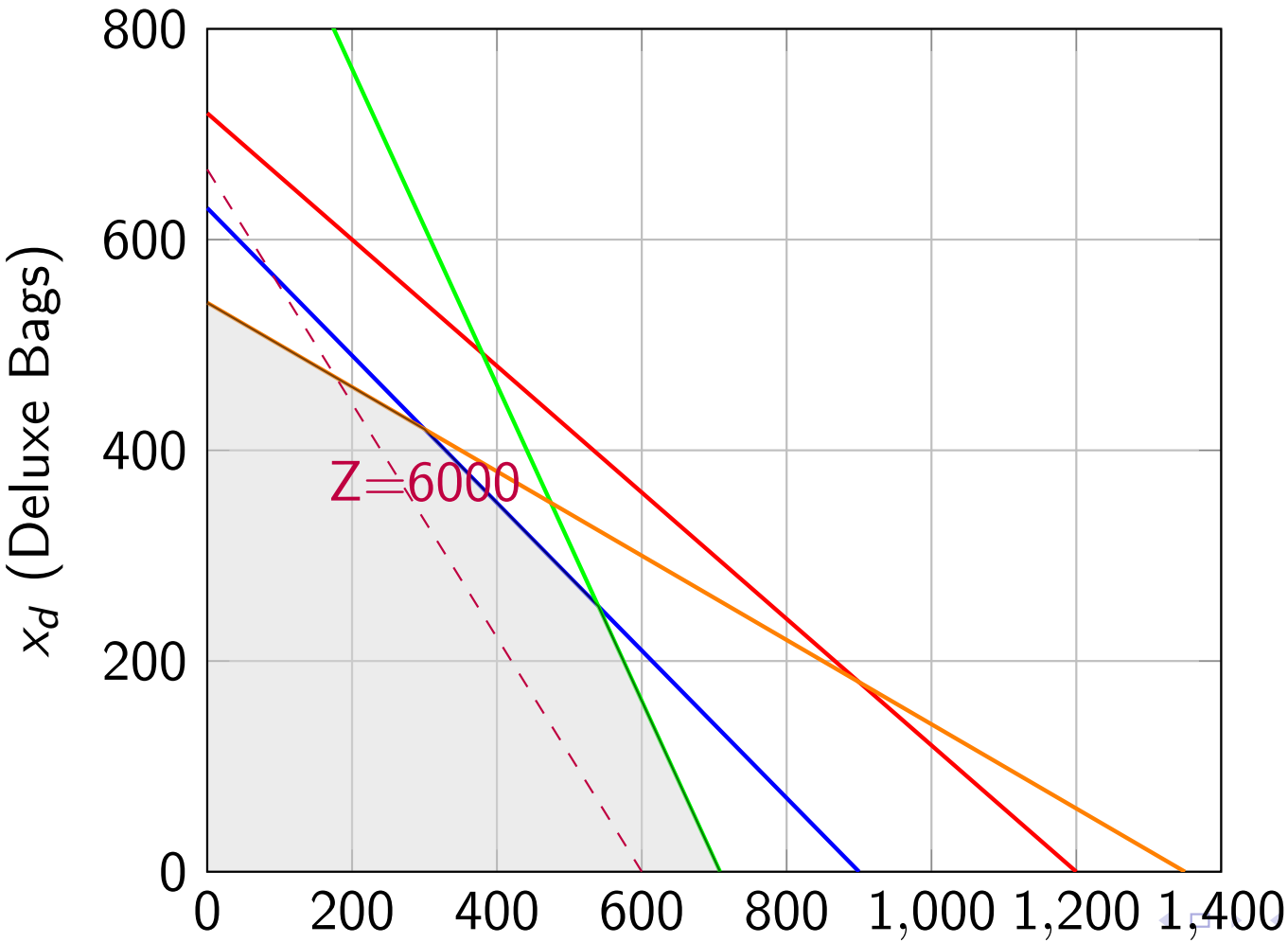
Graphical Solution – Step 8: Objective Function at Low Profit

Objective: $Z = 10x_s + 9x_d = 4000$ Line:
$x_d = \frac{4000}{9} - \frac{10}{9}x_s \approx 444.44 - 1.111x_s$



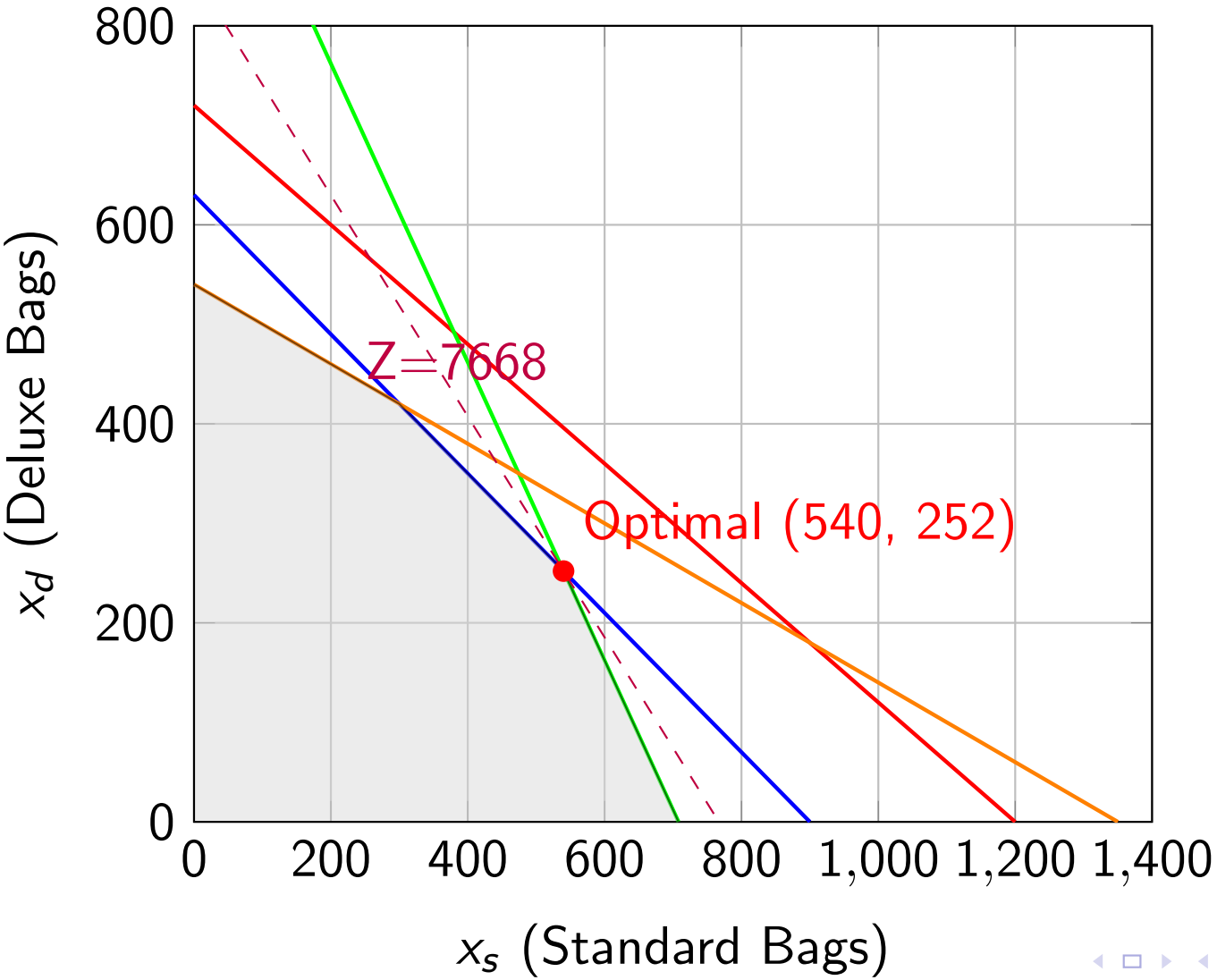
Graphical Solution – Step 9: Objective Function at Higher Profit

Objective: $Z = 10x_s + 9x_d = 6000$ Line:
$x_d = \frac{6000}{9} - \frac{10}{9}x_s \approx 666.67 - 1.111x_s$



Graphical Solution – Step 10: Optimal Objective Function

Objective: $Z = 10x_s + 9x_d = 7668$ Line: $x_d = \frac{7668}{9} - \frac{10}{9}x_s \approx 852 - 1.111x_s$
The line touches the feasible region at the optimal point (540, 252).



Blending Example: MediBlend

Goal: Find the cheapest way to produce today's batch of MediBlend by mixing four chemicals.

The Rules & Constraints:

- **Batch Size:** Must produce *at least* 1,000 lbs total.
- **Ingredient Minimums:** The final mix must contain at least 8% of A, 4% of B, and 2% of C (by weight).
- **Supply Constraint:** Must use *at least* 100 lbs of Chemical 2.

The Data:

Chemical	Cost/lb	Ingredient A	Ingredient B	Ingredient C
Chemical 1	\$8	0.03	0.02	0.01
Chemical 2	\$10	0.06	0.04	0.01
Chemical 3	\$11	0.10	0.03	0.04
Chemical 4	\$14	0.12	0.09	0.04

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\min_{x_1,x_2,x_3,x_4} \quad 8x_1 + 10x_2 + 11x_3 + 14x_4$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned} \min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\ \text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \end{aligned}$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned} \min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\ \text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \\ & 0.03x_1 + 0.06x_2 + 0.10x_3 + 0.12x_4 \geq 0.08(x_1 + x_2 + x_3 + x_4) \end{aligned}$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned} \min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\ \text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \\ & 0.03x_1 + 0.06x_2 + 0.10x_3 + 0.12x_4 \geq 0.08(x_1 + x_2 + x_3 + x_4) \\ & 0.02x_1 + 0.04x_2 + 0.03x_3 + 0.09x_4 \geq 0.04(x_1 + x_2 + x_3 + x_4) \end{aligned}$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned}
\min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\
\text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \\
& 0.03x_1 + 0.06x_2 + 0.10x_3 + 0.12x_4 \geq 0.08(x_1 + x_2 + x_3 + x_4) \\
& 0.02x_1 + 0.04x_2 + 0.03x_3 + 0.09x_4 \geq 0.04(x_1 + x_2 + x_3 + x_4) \\
& 0.01x_1 + 0.01x_2 + 0.04x_3 + 0.04x_4 \geq 0.02(x_1 + x_2 + x_3 + x_4)
\end{aligned}$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned}
\min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\
\text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \\
& 0.03x_1 + 0.06x_2 + 0.10x_3 + 0.12x_4 \geq 0.08(x_1 + x_2 + x_3 + x_4) \\
& 0.02x_1 + 0.04x_2 + 0.03x_3 + 0.09x_4 \geq 0.04(x_1 + x_2 + x_3 + x_4) \\
& 0.01x_1 + 0.01x_2 + 0.04x_3 + 0.04x_4 \geq 0.02(x_1 + x_2 + x_3 + x_4) \\
& x_2 \geq 100
\end{aligned}$$

Blending Example: LP Formulation

Let x_1, x_2, x_3, x_4 denote the pounds of chemicals (1,2,3,4) used to produce today's batch. The objective is to minimize total cost subject to meeting the minimum batch size, the ingredient percentage requirements, the minimum usage of chemical 2, and nonnegativity:

$$\begin{aligned}
\min_{x_1,x_2,x_3,x_4} \quad & 8x_1 + 10x_2 + 11x_3 + 14x_4 \\
\text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 \\
& 0.03x_1 + 0.06x_2 + 0.10x_3 + 0.12x_4 \geq 0.08(x_1 + x_2 + x_3 + x_4) \\
& 0.02x_1 + 0.04x_2 + 0.03x_3 + 0.09x_4 \geq 0.04(x_1 + x_2 + x_3 + x_4) \\
& 0.01x_1 + 0.01x_2 + 0.04x_3 + 0.04x_4 \geq 0.02(x_1 + x_2 + x_3 + x_4) \\
& x_2 \geq 100 \\
& x_1, x_2, x_3, x_4 \geq 0
\end{aligned}$$

Blending Example: Classical LP Formulation

In classical Linear Programming (and for many solvers), we rearrange our equations so that all variables are on the left side of the inequality, and all constants are on the right side.

By factoring out the variables from the right side of our ingredient constraints, our model simplifies to:

$$\min_{x_1,x_2,x_3,x_4} \quad 8x_1 + 10x_2 + 11x_3 + 14x_4$$

$$\begin{aligned}
\text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 \geq 1000 && \text{(Batch Size)} \\
& -0.05x_1 - 0.02x_2 + 0.02x_3 + 0.04x_4 \geq 0 && \text{(Ingredient A)} \\
& -0.02x_1 - 0.01x_3 + 0.05x_4 \geq 0 && \text{(Ingredient B)} \\
& -0.01x_1 - 0.01x_2 + 0.02x_3 + 0.02x_4 \geq 0 && \text{(Ingredient C)} \\
& x_2 \geq 100 && \text{(Chemical 2 Min)} \\
& x_1, x_2, x_3, x_4 \geq 0
\end{aligned}$$

Blending Example Solution (PuLP)

```
import pulp as pl

# Create the problem
prob = pl.LpProblem("MediBlend_Blending", pl.LpMinimize)

# Define variables directly into the model
x1 = prob.add_variable("Chemical_1", lowBound=0)
x2 = prob.add_variable("Chemical_2", lowBound=0)
x3 = prob.add_variable("Chemical_3", lowBound=0)
x4 = prob.add_variable("Chemical_4", lowBound=0)

# Objective function
prob += 8 * x1 + 10 * x2 + 11 * x3 + 14 * x4, "Minimize_Cost"

# Constraints
prob += x1 + x2 + x3 + x4 >= 1000, "Batch_Size"
prob += 0.03 * x1 + 0.06 * x2 + 0.10 * x3 + 0.12 * x4 >= 0.08 * (x1 + x2 + x3 + x4), "
    Ingredient_A"
prob += 0.02 * x1 + 0.04 * x2 + 0.03 * x3 + 0.09 * x4 >= 0.04 * (x1 + x2 + x3 + x4), "
    Ingredient_B"
prob += 0.01 * x1 + 0.01 * x2 + 0.04 * x3 + 0.04 * x4 >= 0.02 * (x1 + x2 + x3 + x4), "
    Ingredient_C"
prob += x2 >= 100, "Chemical_2_Min"

# Solve the problem
res = prob.solve()

# Print results
print("Status:", res.status.name)
print("Optimal Solution:")
print(f"Chemical 1 = {x1.varValue} lbs")
print(f"Chemical 2 = {x2.varValue} lbs")
print(f"Chemical 3 = {x3.varValue} lbs")
print(f"Chemical 4 = {x4.varValue} lbs")
print(f"Minimum Cost = ${pl.value(prob.objective)}")
```

Work Scheduling Example

A post office requires different numbers of full-time employees on different days of the week. The number of full-time employees needed on each day is as follows:

Day	Number of Employees
Monday	7
Tuesday	3
Wednesday	5
Thursday	9
Friday	4
Saturday	6
Sunday	1

Table: Weekly Employee Requirements

Union rules state that each full-time employee must work five consecutive days and then receive two days off. For example, an employee who works Monday to Friday must be off Saturday and Sunday. The post office wants to meet its daily requirements using only full-time employees. Formulate an LP that the post office can use to minimize the number of full-time employees that must be hired.

Work Scheduling: Shift Coverage Grid

To formulate the constraints, it helps to visualize which days each shift covers. Employees work 5 consecutive days and then get 2 days off.

Shift Starts	**Mon**	**Tue**	**Wed**	**Thu**	**Fri**	**Sat**	**Sun**
Monday (x_{Mon})	W	W	W	W	W	Off	Off
Tuesday (x_{Tue})	Off	W	W	W	W	W	Off
Wednesday (x_{Wed})	Off	Off	W	W	W	W	W
Thursday (x_{Thu})	W	Off	Off	W	W	W	W
Friday (x_{Fri})	W	W	Off	Off	W	W	W
Saturday (x_{Sat})	W	W	W	Off	Off	W	W
Sunday (x_{Sun})	W	W	W	W	Off	Off	W
Required	**7**	**3**	**5**	**9**	**4**	**6**	**1**

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$\min \quad x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}}$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$\begin{aligned} \min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} \\ \text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 \qquad \text{(Mon)} \end{aligned}$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)} \\
& x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} \geq 9 & \text{(Thu)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)} \\
& x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} \geq 9 & \text{(Thu)} \\
& x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} \geq 4 & \text{(Fri)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)} \\
& x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} \geq 9 & \text{(Thu)} \\
& x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} \geq 4 & \text{(Fri)} \\
& x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} \geq 6 & \text{(Sat)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)} \\
& x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} \geq 9 & \text{(Thu)} \\
& x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} \geq 4 & \text{(Fri)} \\
& x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} \geq 6 & \text{(Sat)} \\
& x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} \geq 1 & \text{(Sun)}
\end{aligned}
$$

Work Scheduling Example: LP Formulation

Let $(x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}})$ denote the number of full-time employees whose 5-day workweek begins on the indicated day (each employee works 5 consecutive days, then has 2 consecutive days off). The goal is to minimize the total number of employees started this week subject to daily staffing requirements and nonnegativity:

$$
\begin{aligned}
\min \quad & x_{\text{Mon}} + x_{\text{Tue}} + x_{\text{Wed}} + x_{\text{Thu}} + x_{\text{Fri}} + x_{\text{Sat}} + x_{\text{Sun}} & \\
\text{s.t.} \quad & x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} \geq 7 & \text{(Mon)} \\
& x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} \geq 3 & \text{(Tue)} \\
& x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} + x_{\text{Sat}} \geq 5 & \text{(Wed)} \\
& x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} + x_{\text{Sun}} \geq 9 & \text{(Thu)} \\
& x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} + x_{\text{Mon}} \geq 4 & \text{(Fri)} \\
& x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} + x_{\text{Tue}} \geq 6 & \text{(Sat)} \\
& x_{\text{Sun}} + x_{\text{Sat}} + x_{\text{Fri}} + x_{\text{Thu}} + x_{\text{Wed}} \geq 1 & \text{(Sun)} \\
& x_{\text{Mon}}, x_{\text{Tue}}, x_{\text{Wed}}, x_{\text{Thu}}, x_{\text{Fri}}, x_{\text{Sat}}, x_{\text{Sun}} \geq 0. &
\end{aligned}
$$

Work Scheduling Solution (PuLP) (1/2)

```
import pulp as pl

# Create the problem
prob = pl.LpProblem("Post_Office_Scheduling", pl.LpMinimize)

# Define variables directly into the model
x_mon = prob.add_variable("x_Mon", lowBound=0, cat='Integer')
x_tue = prob.add_variable("x_Tue", lowBound=0, cat='Integer')
x_wed = prob.add_variable("x_Wed", lowBound=0, cat='Integer')
x_thu = prob.add_variable("x_Thu", lowBound=0, cat='Integer')
x_fri = prob.add_variable("x_Fri", lowBound=0, cat='Integer')
x_sat = prob.add_variable("x_Sat", lowBound=0, cat='Integer')
x_sun = prob.add_variable("x_Sun", lowBound=0, cat='Integer')

# Objective function
prob += x_mon + x_tue + x_wed + x_thu + x_fri + x_sat + x_sun, "Total_Employees"

# Constraints
prob += x_mon + x_sun + x_sat + x_fri + x_thu >= 7, "Monday"
prob += x_tue + x_mon + x_sun + x_sat + x_fri >= 3, "Tuesday"
prob += x_wed + x_tue + x_mon + x_sun + x_sat >= 5, "Wednesday"
prob += x_thu + x_wed + x_tue + x_mon + x_sun >= 9, "Thursday"
prob += x_fri + x_thu + x_wed + x_tue + x_mon >= 4, "Friday"
prob += x_sat + x_fri + x_thu + x_wed + x_tue >= 6, "Saturday"
prob += x_sun + x_sat + x_fri + x_thu + x_wed >= 1, "Sunday"
```

Work Scheduling Solution (PuLP) (2/2)

```
# Solve the problem
res = prob.solve()

# Print results
print("Status:", res.status.name)
print("Optimal Solution:")
print(f"Employees starting Mon = {x_mon.varValue}")
print(f"Employees starting Tue = {x_tue.varValue}")
print(f"Employees starting Wed = {x_wed.varValue}")
print(f"Employees starting Thu = {x_thu.varValue}")
print(f"Employees starting Fri = {x_fri.varValue}")
print(f"Employees starting Sat = {x_sat.varValue}")
print(f"Employees starting Sun = {x_sun.varValue}")
print(f"Minimum Employees = {pl.value(prob.objective)}")
```

Blending Example II

The Java Junkie Coffee Shop blends coffee on the premises for its customers. It sells three basic blends in 1-pound bags: Special, Mountain Dark, and Mill Regular. It uses two different types of beans to produce the blends: Brazilian and mild. The shop used the following blend recipe requirements.

BLEND	MIX REQUIREMENTS	SELLING PRICE ($/LB)
Special	At least 40% Brazilian No more 30% mild	6.50
Dark	At least 60% Brazilian No more than 10% mild	5.25
Regular	No more than 60% mild At least 30% Brazilian	3.75

The cost of Brazilian coffee beans are $2.00 per pound and the cost of mild is $1.25 per pound. Every week the shop can buy up-to 210 pounds of Brazilian coffee beans, and 300 pounds of mild coffee beans. The shop wants to know the amount of each blend it should prepare each week and how much of each type of coffee beans to include in each blend in order to maximize profits. Assume the weekly demand to all three blends always exceeds the store's supply.

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$\max \quad 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR})$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$\begin{aligned}\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\ & - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR})\end{aligned}$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}),
\end{aligned}$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}), \\
& \text{Dark: } x_{bD} \geq 0.60(x_{bD} + x_{mD}), \quad x_{mD} \leq 0.10(x_{bD} + x_{mD}),
\end{aligned}$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& \quad - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}), \\
& \text{Dark: } x_{bD} \geq 0.60(x_{bD} + x_{mD}), \quad x_{mD} \leq 0.10(x_{bD} + x_{mD}), \\
& \text{Regular: } x_{mR} \leq 0.60(x_{bR} + x_{mR}), \quad x_{bR} \geq 0.30(x_{bR} + x_{mR}),
\end{aligned}$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$
\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& \quad - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}), \\
& \text{Dark: } x_{bD} \geq 0.60(x_{bD} + x_{mD}), \quad x_{mD} \leq 0.10(x_{bD} + x_{mD}), \\
& \text{Regular: } x_{mR} \leq 0.60(x_{bR} + x_{mR}), \quad x_{bR} \geq 0.30(x_{bR} + x_{mR}), \\
& \text{Brazilian availability: } x_{bS} + x_{bD} + x_{bR} \leq 210,
\end{aligned}
$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$
\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& \quad - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}), \\
& \text{Dark: } x_{bD} \geq 0.60(x_{bD} + x_{mD}), \quad x_{mD} \leq 0.10(x_{bD} + x_{mD}), \\
& \text{Regular: } x_{mR} \leq 0.60(x_{bR} + x_{mR}), \quad x_{bR} \geq 0.30(x_{bR} + x_{mR}), \\
& \text{Brazilian availability: } x_{bS} + x_{bD} + x_{bR} \leq 210, \\
& \text{Mild availability: } x_{mS} + x_{mD} + x_{mR} \leq 300,
\end{aligned}
$$

Blending Example II: LP with Double-Indexed Variables

Let (x_{bS}, x_{bD}, x_{bR}) denote pounds of Brazilian beans used in Special, Dark, and Regular, and (x_{mS}, x_{mD}, x_{mR}) denote pounds of mild beans used in those blends. The objective maximizes profit (revenue minus bean costs), with recipe, availability, and nonnegativity constraints:

$$
\begin{aligned}
\max \quad & 6.50(x_{bS} + x_{mS}) + 5.25(x_{bD} + x_{mD}) + 3.75(x_{bR} + x_{mR}) \\
& - 2.00(x_{bS} + x_{bD} + x_{bR}) - 1.25(x_{mS} + x_{mD} + x_{mR}) \\
\text{s.t.} \quad & \text{Special: } x_{bS} \geq 0.40(x_{bS} + x_{mS}), \quad x_{mS} \leq 0.30(x_{bS} + x_{mS}), \\
& \text{Dark: } x_{bD} \geq 0.60(x_{bD} + x_{mD}), \quad x_{mD} \leq 0.10(x_{bD} + x_{mD}), \\
& \text{Regular: } x_{mR} \leq 0.60(x_{bR} + x_{mR}), \quad x_{bR} \geq 0.30(x_{bR} + x_{mR}), \\
& \text{Brazilian availability: } x_{bS} + x_{bD} + x_{bR} \leq 210, \\
& \text{Mild availability: } x_{mS} + x_{mD} + x_{mR} \leq 300, \\
& x_{bS}, x_{bD}, x_{bR}, x_{mS}, x_{mD}, x_{mR} \geq 0.
\end{aligned}
$$

Equivalently, the recipe constraints may be written in purely linear form:

$$
\begin{aligned}
& 0.60x_{bS} - 0.40x_{mS} \geq 0, \quad -0.30x_{bS} + 0.70x_{mS} \leq 0, \\
& 0.40x_{bD} - 0.60x_{mD} \geq 0, \quad -0.10x_{bD} + 0.90x_{mD} \leq 0, \\
& -0.60x_{bR} + 0.40x_{mR} \leq 0, \quad 0.70x_{bR} - 0.30x_{mR} \geq 0.
\end{aligned}
$$

Coffee Blending Solution (PuLP) (1/2)

```
import pulp as pl

# Create the problem
prob = pl.LpProblem("Java_Junkie_Coffee", pl.LpMaximize)

# Define variables directly into the model
x_bS = prob.add_variable("Brazilian_Special", lowBound=0)
x_bD = prob.add_variable("Brazilian_Dark", lowBound=0)
x_bR = prob.add_variable("Brazilian_Regular", lowBound=0)
x_mS = prob.add_variable("Mild_Special", lowBound=0)
x_mD = prob.add_variable("Mild_Dark", lowBound=0)
x_mR = prob.add_variable("Mild_Regular", lowBound=0)

# Objective function
prob += (6.50 * (x_bS + x_mS) + 5.25 * (x_bD + x_mD) + 3.75 * (x_bR + x_mR)
         - 2.00 * (x_bS + x_bD + x_bR) - 1.25 * (x_mS + x_mD + x_mR)), "Maximize_Profit"

# Constraints
# Special blend constraints
prob += x_bS >= 0.40 * (x_bS + x_mS), "Special_Brazilian_Min"
prob += x_mS <= 0.30 * (x_bS + x_mS), "Special_Mild_Max"

# Dark blend constraints
prob += x_bD >= 0.60 * (x_bD + x_mD), "Dark_Brazilian_Min"
prob += x_mD <= 0.10 * (x_bD + x_mD), "Dark_Mild_Max"

# Regular blend constraints
prob += x_mR <= 0.60 * (x_bR + x_mR), "Regular_Mild_Max"
prob += x_bR >= 0.30 * (x_bR + x_mR), "Regular_Brazilian_Min"

# Availability constraints
prob += x_bS + x_bD + x_bR <= 210, "Brazilian_Availability"
prob += x_mS + x_mD + x_mR <= 300, "Mild_Availability"
```

Coffee Blending Solution (PuLP) (2/2)

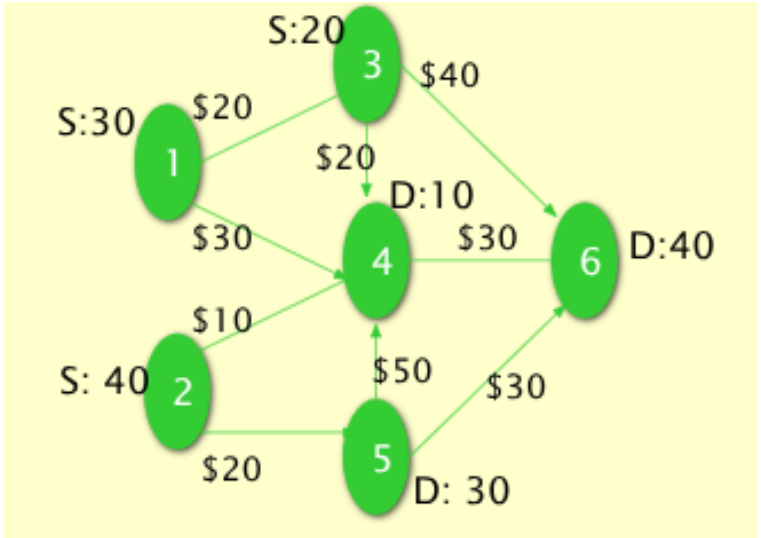
```
# Solve the problem
res = prob.solve()

# Print results
print("Status:", res.status.name)
print("Optimal Solution:")
print(f"Brazilian in Special = {x_bS.varValue} lbs")
print(f"Brazilian in Dark = {x_bD.varValue} lbs")
print(f"Brazilian in Regular = {x_bR.varValue} lbs")
print(f"Mild in Special = {x_mS.varValue} lbs")
print(f"Mild in Dark = {x_mD.varValue} lbs")
print(f"Mild in Regular = {x_mR.varValue} lbs")
print(f"Maximum Profit = ${pl.value(prob.objective)}")
```

Network Model: Transportation Problem

A furniture manufacturer, Furniture Max, has three warehouses (1, 2, 3), and three retail stores (4, 5, 6). The network diagram indicates the shipping routes and shipping costs:

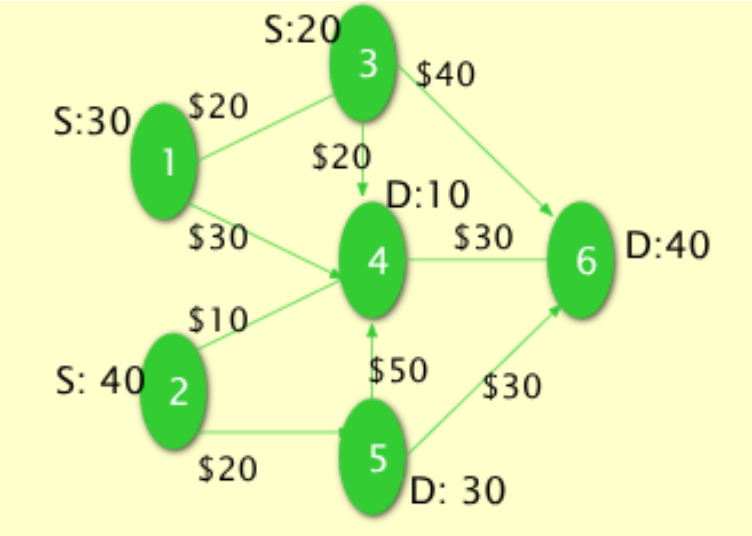
- **Arc values:** Per-unit shipping costs between cities.
- **Nodes 1, 2, 3:** Warehouse supply capacities.
- **Nodes 4, 5, 6:** Retail store demand requirements.



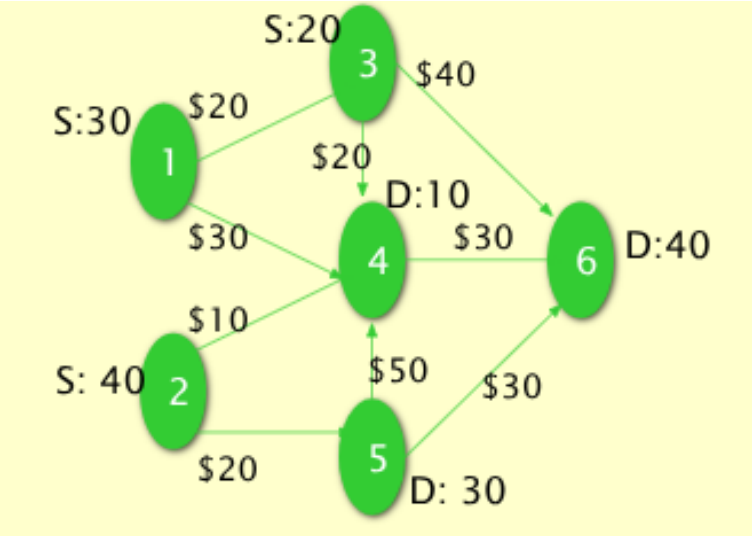
Goal: Formulate an LP to determine the least costly shipping plan for Furniture Max.

Transportation Problem

Decisions: x_{ij} = units shipped from node i to node j



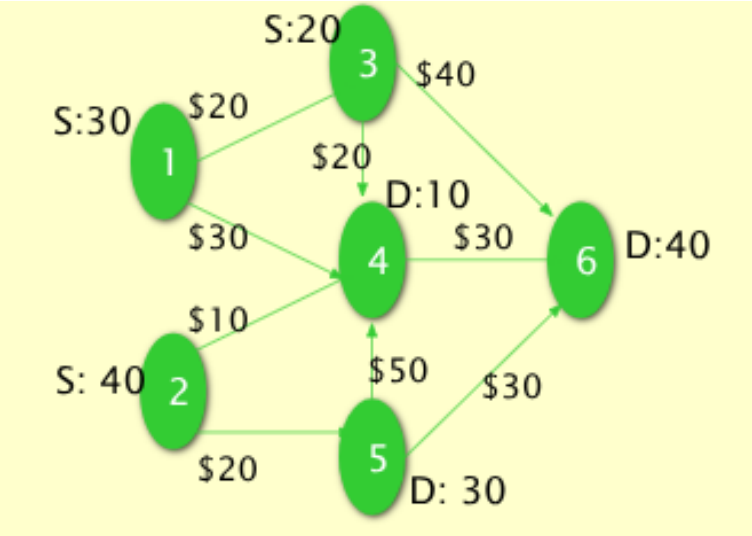
Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Transportation Problem



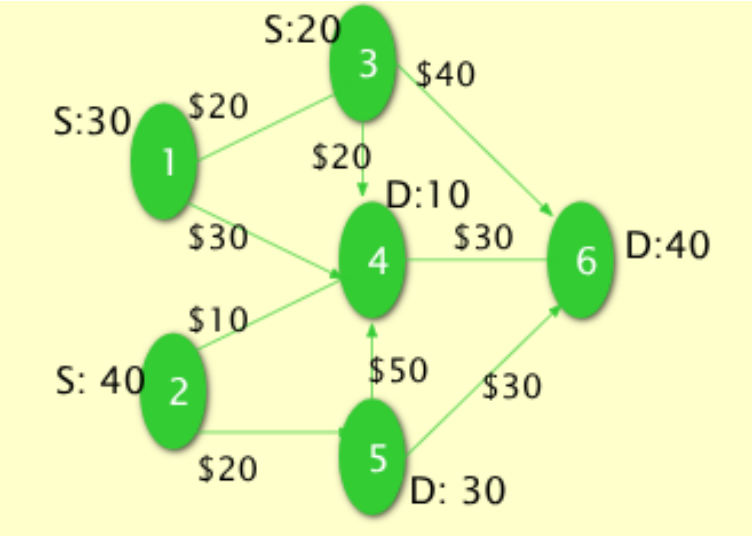
Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

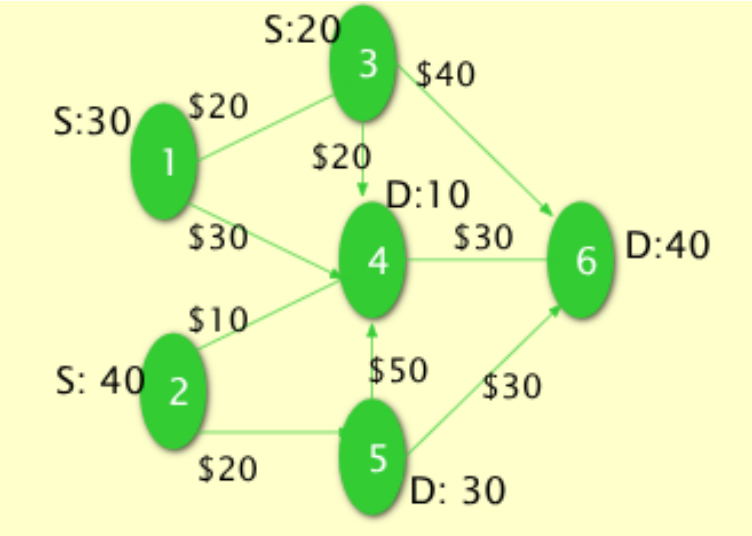
Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

Node 2: $x_{24} + x_{25} \leq 40$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

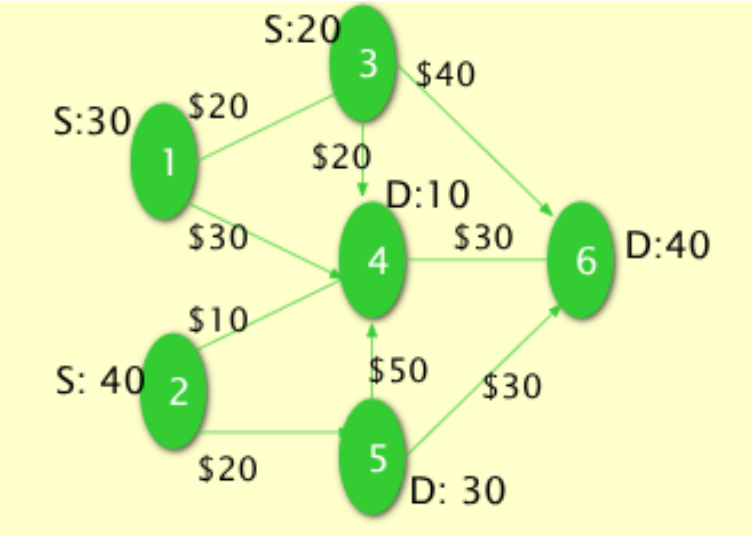
Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

Node 2: $x_{24} + x_{25} \leq 40$

Node 3: $x_{36} + x_{34} - x_{13} \leq 20$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

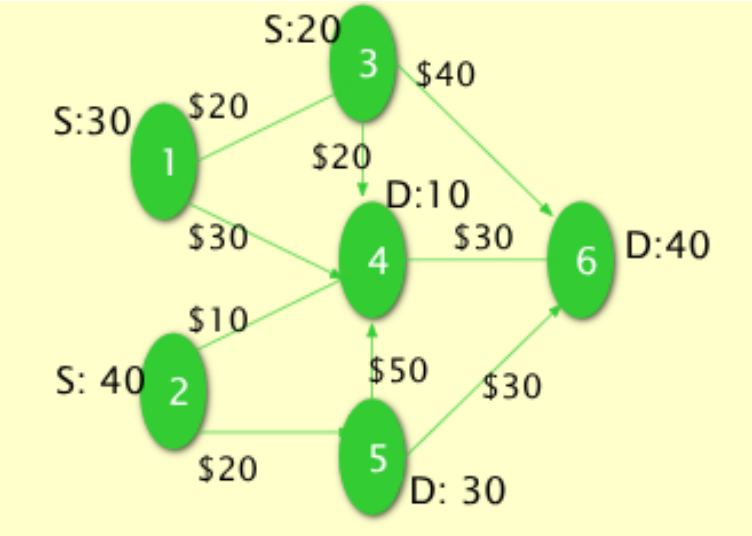
Node 1: $x_{13} + x_{14} \leq 30$

Node 2: $x_{24} + x_{25} \leq 40$

Node 3: $x_{36} + x_{34} - x_{13} \leq 20$

Node 4: $x_{14} + x_{24} + x_{34} + x_{54} - x_{46} \geq 10$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

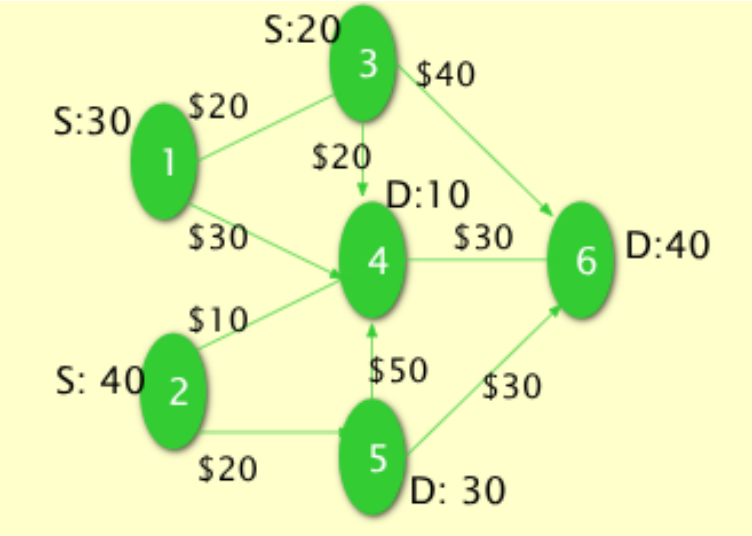
Node 2: $x_{24} + x_{25} \leq 40$

Node 3: $x_{36} + x_{34} - x_{13} \leq 20$

Node 4: $x_{14} + x_{24} + x_{34} + x_{54} - x_{46} \geq 10$

Node 5: $x_{25} - x_{54} - x_{56} \geq 30$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

Node 2: $x_{24} + x_{25} \leq 40$

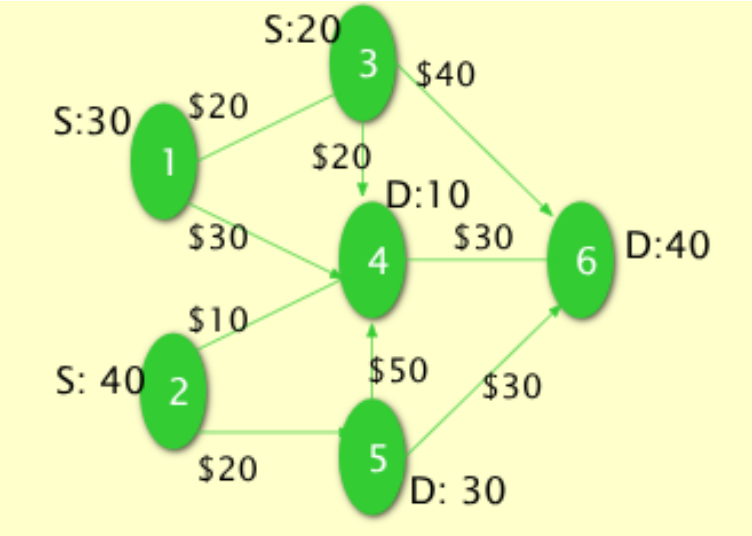
Node 3: $x_{36} + x_{34} - x_{13} \leq 20$

Node 4: $x_{14} + x_{24} + x_{34} + x_{54} - x_{46} \geq 10$

Node 5: $x_{25} - x_{54} - x_{56} \geq 30$

Node 6: $x_{36} + x_{56} + x_{46} \geq 40$

Transportation Problem



Decisions: x_{ij} = units shipped from node i to node j

Objective: min $20x_{13} + 30x_{14} + \cdots + 30x_{56}$

Constraints:

Node 1: $x_{13} + x_{14} \leq 30$

Node 2: $x_{24} + x_{25} \leq 40$

Node 3: $x_{36} + x_{34} - x_{13} \leq 20$

Node 4: $x_{14} + x_{24} + x_{34} + x_{54} - x_{46} \geq 10$

Node 5: $x_{25} - x_{54} - x_{56} \geq 30$

Node 6: $x_{36} + x_{56} + x_{46} \geq 40$

Non-negativity: $x_{ij} \geq 0$

Transportation Problem Solution (PuLP) (1/2)

```
import pulp as pl

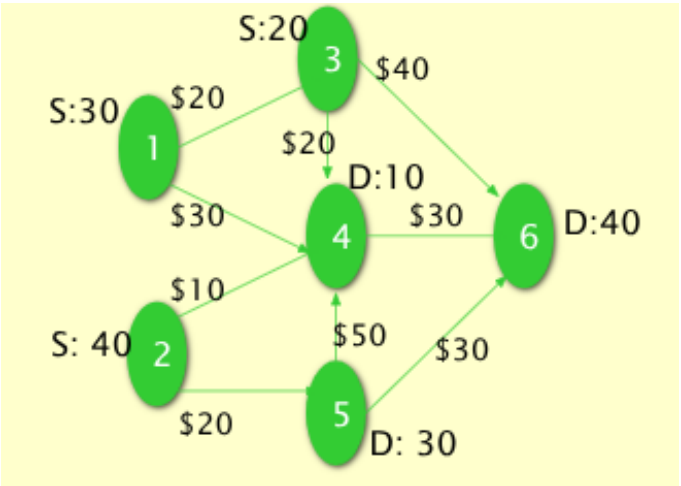
# Create the problem
prob = pl.LpProblem("Furniture_Max", pl.LpMinimize)

# Define variables directly into the model
x13 = prob.add_variable("x13", lowBound=0)
x14 = prob.add_variable("x14", lowBound=0)
x24 = prob.add_variable("x24", lowBound=0)
x25 = prob.add_variable("x25", lowBound=0)
x34 = prob.add_variable("x34", lowBound=0)
x36 = prob.add_variable("x36", lowBound=0)
x46 = prob.add_variable("x46", lowBound=0)
x54 = prob.add_variable("x54", lowBound=0)
x56 = prob.add_variable("x56", lowBound=0)

# Objective function
prob += (20*x13 + 30*x14 + 10*x24 + 20*x25 + 30*x34
         + 40*x36 + 30*x46 + 50*x54 + 30*x56, "Min_Cost
              ")

# Supply constraints
prob += x13 + x14 <= 30, "Node_1_Supply"
prob += x24 + x25 <= 40, "Node_2_Supply"
prob += x34 + x36 - x13 <= 20, "Node_3_Supply"

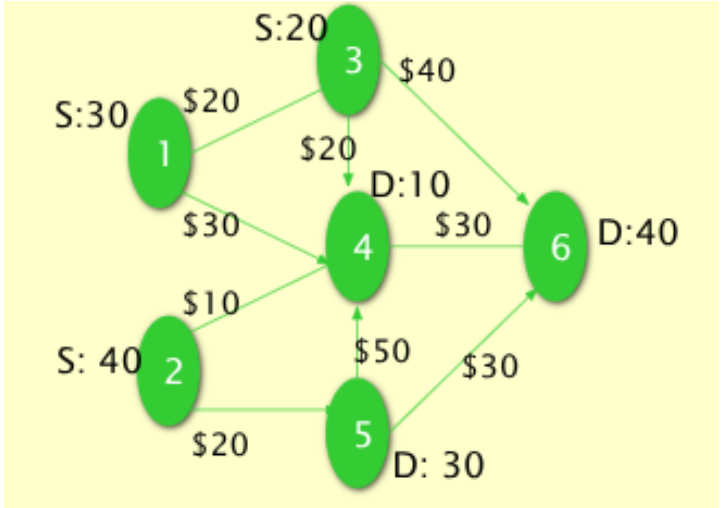
# Demand constraints
prob += x14 + x24 + x34 + x54 - x46 >= 10, "
     Node_4_Demand"
prob += x25 - x54 - x56 >= 30, "Node_5_Demand"
prob += x36 + x56 + x46 >= 40, "Node_6_Demand"
```



Transportation Problem Solution (PuLP) (2/2)

```
# Solve the problem
res = prob.solve()

# Print results
print("Status:", res.status.name)
print("Optimal Solution:")
print(f"x13 = {x13.varValue}")
print(f"x14 = {x14.varValue}")
print(f"x24 = {x24.varValue}")
print(f"x25 = {x25.varValue}")
print(f"x34 = {x34.varValue}")
print(f"x36 = {x36.varValue}")
print(f"x46 = {x46.varValue}")
print(f"x54 = {x54.varValue}")
print(f"x56 = {x56.varValue}")
print(f"Minimum Cost = ${pl.value(prob.objective)}")
```



Network Models: The Assignment Problem

Resource Allocation & Project Staffing

Consulting Staffing Problem

A firm must assign 4 Senior Project Leaders to 4 Client Projects (1-to-1 matching).

Leader	**P1**	**P2**	**P3**	**P4**
Adams	38	75	44	27
Baker	34	76	43	25
Clark	41	71	41	26
Davis	33	80	45	30

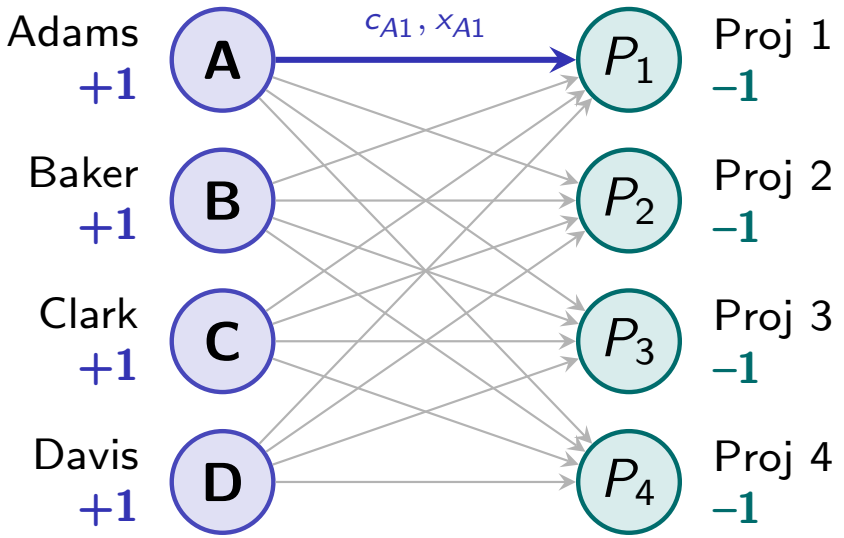
Values represent completion times in hours.

Model Characteristics

- **1-to-1 Matching:** Bipartite assignment between n agents and n tasks.
- **Objective:** Minimize total consulting hours across all projects.
- **Combinatorial Scale:** $4! = 24$ permutations. For $n = 10$, $10! \approx 3.63 \times 10^6$.
- **LP Advantage:** Solves in milliseconds without brute-force search!

The Assignment Problem as a Network Flow

Bipartite Graph Formulation



Decision Arcs: $x_{ij} \in \{0, 1\}$ with unit cost c_{ij} hours

Network Flow Properties

- **Specialized Transportation Model:** Bipartite network with source nodes I (supply $s_i = 1$) and sink nodes J (demand $d_j = 1$).
- **Balanced Flow Conservation:** $\sum_{i=1}^{4} s_i = \sum_{j=1}^{4} d_j = 4$.
- **Unit Capacity Arcs:** Flow across arc (i, j) is either 0 or 1.
- **Decision Variable:** $x_{ij} = 1$ if leader $i \rightarrow$ project j; 0 otherwise.

Assignment Problem: Mathematical Formulation

Binary Integer Program vs. Continuous Linear Program

Optimization Model

$$\min \; Z = \sum_{i=1}^{4} \sum_{j=1}^{4} c_{ij} x_{ij}$$

Subject to:

- $\sum_{j=1}^{4} x_{ij} = 1 \quad \forall i$ (1 proj / leader)
- $\sum_{i=1}^{4} x_{ij} = 1 \quad \forall j$ (1 leader / proj)
- $x_{ij} \geq 0 \quad \forall i, j$ (Non-negativity)

Total Unimodularity (TUM)

- **TUM Incidence Matrix:** Bipartite node-arc matrix is totally unimodular.
- **Guaranteed Integrality:** Integer RHS (± 1) guarantees binary solutions ($x_{ij}^* \in \{0, 1\}$).
- **Solves as LP:** Continuous Simplex guarantees integer solutions instantly!

Assignment Problem in Python: PuLP (1/2)

Problem Definition & Parameter Setup

```
import pulp as pl

# 1. Initialize problem
prob = pl.LpProblem("Project_Assignment", pl.LpMinimize)

# 2. Sets and completion hours matrix
leaders = ['Adams', 'Baker', 'Clark', 'Davis']
projects = ['P1', 'P2', 'P3', 'P4']
costs = {
    'Adams': {'P1': 38, 'P2': 75, 'P3': 44, 'P4': 27},
    'Baker': {'P1': 34, 'P2': 76, 'P3': 43, 'P4': 25},
    'Clark': {'P1': 41, 'P2': 71, 'P3': 41, 'P4': 26},
    'Davis': {'P1': 33, 'P2': 80, 'P3': 45, 'P4': 30}
}

# 3. Decision variables: x[i, j] in {0, 1}
x = prob.add_variable_dicts("Assign", (leaders, projects),
                            lowBound=0, upBound=1, cat='Integer')

# 4. Objective: Minimize total hours
prob += pl.lpSum(costs[i][j] * x[i][j] for i in leaders for j in
    projects)
```

Implementation Mechanics

- `prob.add_variable_dicts()`: Creates 16 binary variables x_{ij} over leaders $\times$ projects.
- `pl.lpSum()`: Compactly generates the double summation $\sum_i \sum_j c_{ij} x_{ij}$.

Assignment Problem in Python: PuLP (2/2)

Constraints, Solution & Optimal Staffing Schedule

```
# Each leader gets 1 project
for i in leaders:
    prob += pl.lpSum(x[i][j] for j
        in projects) == 1

# Each project gets 1 leader
for j in projects:
    prob += pl.lpSum(x[i][j] for i
        in leaders) == 1

# Solve optimization model
res = prob.solve()
print("Status:", res.status.name
    )
print(f"Min Hours = {pl.value(
    prob.objective)}")
```

Optimal Schedule ($Z^* = 173$ h)

Leader	Project	Hours
Adams	Project 3	44 h
Baker	Project 4	25 h
Clark	Project 2	71 h
Davis	Project 1	33 h

Managerial Insight

- **Global Optimization:** Assigning Baker to P4 (25 h) frees Adams for P3 (44 h).
- **Bottleneck Match:** Clark must lead P2 (71 h); others incur a 4–9 h penalty.

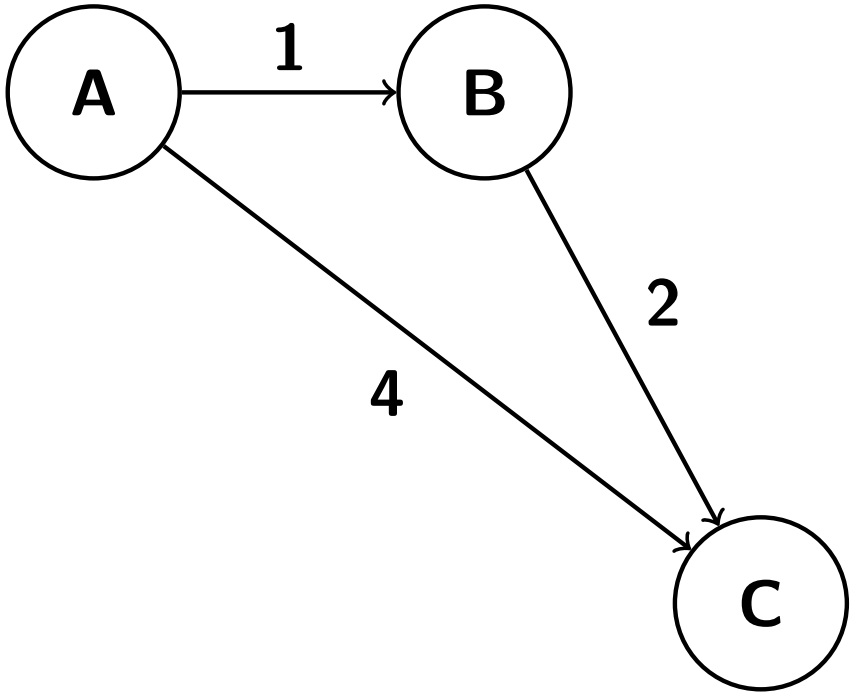
The Shortest Path Problem

LP and an Heuristic

- Using a toy example

The Shortest Path Problem: An Example

Find the shortest path from A to C.



Edges with costs:

- A to B: **1**
- A to C: **4**
- B to C: **2**

Problem Statement

- The shortest path problem seeks the path from source (A) to sink (C) with minimal total cost.
- Traditional algorithms: Dijkstra, Bellman-Ford.
- Here, we formulate it as a Linear Program (LP) using network flow.

LP Formulation: Variables and Objective

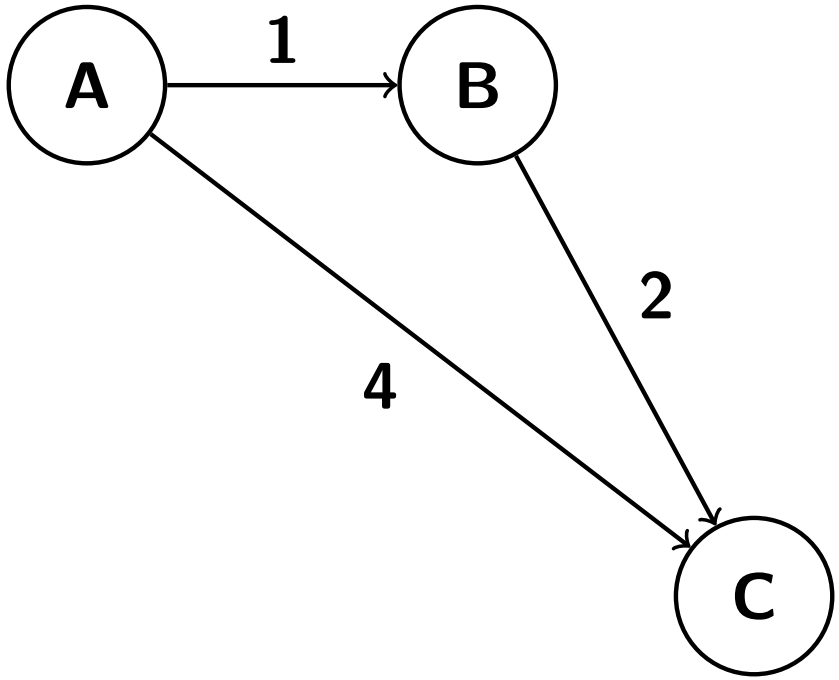
Variables (flow on edges):

- x_{AB}: flow from A to B
- x_{AC}: flow from A to C
- x_{BC}: flow from B to C

Objective: Minimize total cost

$$\min \ 1 \cdot x_{AB} + 4 \cdot x_{AC} + 2 \cdot x_{BC}$$

LP Formulation: Constraints



Flow conservation (net flow out):

- Source A: net out $= 1$ $\quad x_{AB} + x_{AC} = 1$
- Intermediate B: net out $= 0$ $\quad x_{BC} - x_{AB} = 0$
- Sink C: net out $=$ -1 $\quad -(x_{AC} + x_{BC}) = -1$ or $x_{AC} + x_{BC} = 1$

Non-negativity: $x_{AB}, x_{AC}, x_{BC} \geq 0$

Solving with PuLP

```
import pulp as pl

prob = pl.LpProblem("ShortestPath", pl.LpMinimize)

x_ab = prob.add_variable("x_ab", lowBound=0)
x_ac = prob.add_variable("x_ac", lowBound=0)
x_bc = prob.add_variable("x_bc", lowBound=0)

prob += 1*x_ab + 4*x_ac + 2*x_bc

prob += x_ab + x_ac == 1, "Node_A"
prob += x_bc - x_ab == 0, "Node_B"
prob += x_ac + x_bc == 1, "Node_C"

res = prob.solve()
```

Optimal Solution

- $x_{AB} = 1$
- $x_{AC} = 0$
- $x_{BC} = 1$
- Total Cost $= 3$

Path: A $\rightarrow$ B $\rightarrow$ C (cost $1 + 2 = 3$)

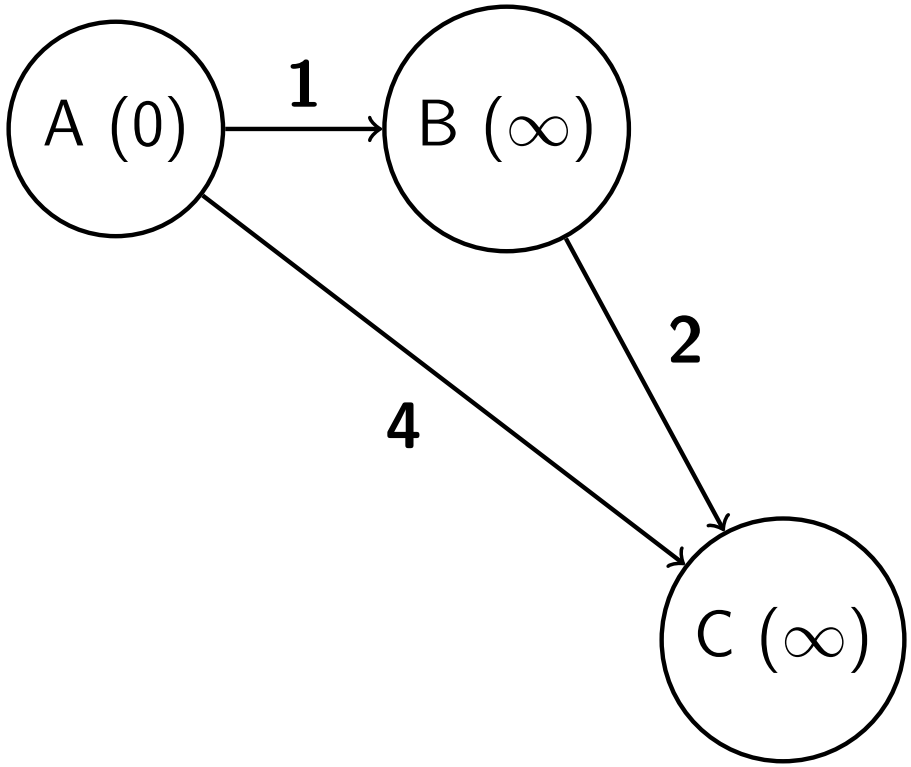
Conclusion

- The LP formulation models the shortest path as a min-cost flow problem with unit supply/demand.
- Advantages: Handles variants (e.g., capacities); solvable with LP solvers.
- Note: For pure shortest path, integer solutions are guaranteed due to network structure.

.

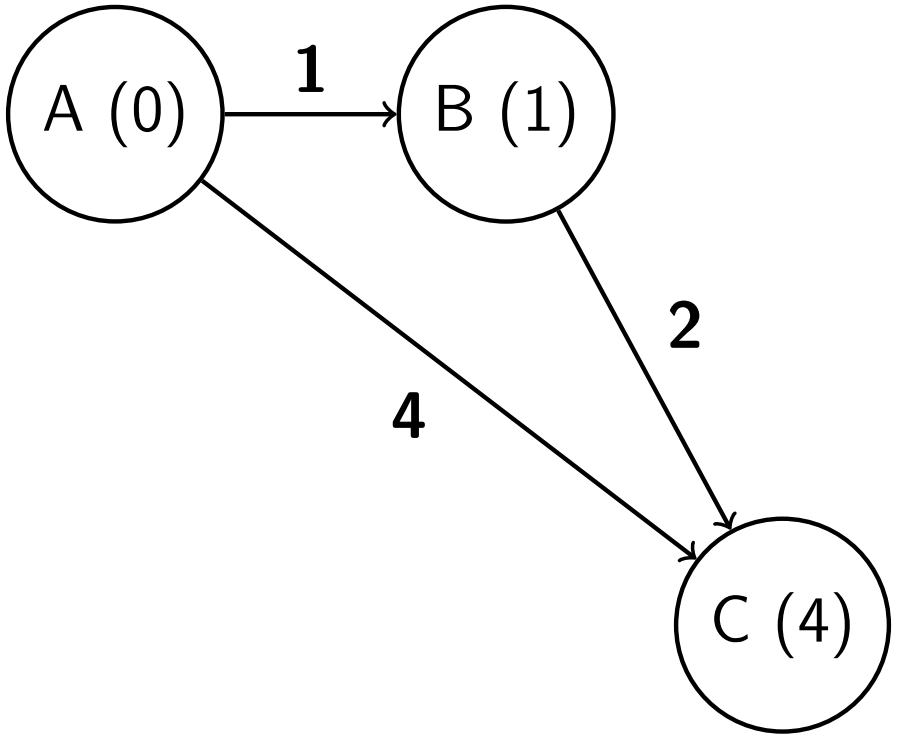
Dijkstra's Algorithm: Step 1 - Initialization

Find the shortest path from A to C.



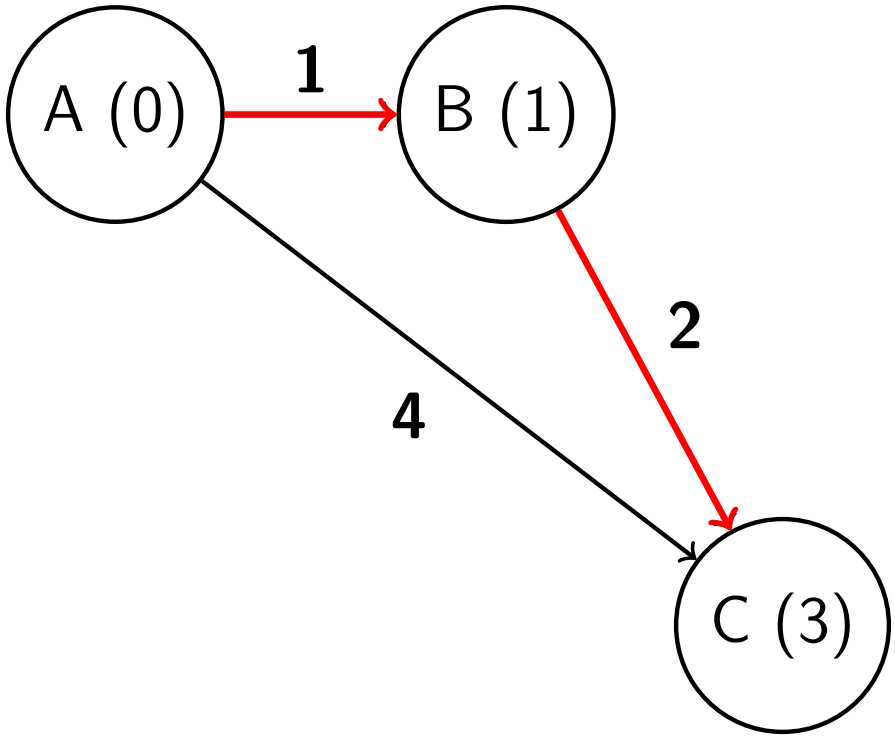
- Start at node A, set distance to 0. - Initialize distances to all other nodes as ∞. - Set A as the current node.

Dijkstra's Algorithm: Step 2 - Update Distances



- Update distances from A: - B: $0 + 1 = 1$ (update from ∞) - C: $0 + 4 = 4$ (update from ∞) - Mark A as visited. - Move to unvisited node with smallest distance: B (1).

Dijkstra's Algorithm: Step 3 - Final Path



- Update distances from B: - C: $\min(4, 1+2) = 3$ (update from 4) - Mark B as visited. - Only unvisited node is C (3), mark as visited. - Shortest path: A $\rightarrow$ B $\rightarrow$ C with total cost 3.

Sensitivity Analysis in Linear Programming

Shadow Prices and Reduced Costs

- Using Par, Inc. Golf Bag Production as Example

Introduction to Sensitivity Analysis

Sensitivity analysis examines how changes in the LP parameters (coefficients in objective, RHS of constraints, etc.) affect the optimal solution.

Why important?

- Helps understand the robustness of the solution.
- Provides managerial insights (e.g., value of resources).
- Identifies critical parameters.

Key concepts:

- Shadow prices (for constraints)
- Reduced costs (for variables)
- Ranges of validity

Recap of Par, Inc. Problem

Decision variables:

Let x_s = number of standard golf bags produced per week,
x_d = number of deluxe golf bags produced per week.
Both $x_s, x_d \geq 0$.

Objective function:

Maximize weekly profit:

$$\max \ Z = 10x_s + 9x_d$$

Subject to capacity constraints:

$$\begin{aligned}
\frac{7}{10}x_s + x_d &\leq 630 && \text{(Cutting \& Dyeing)} \\
\frac{1}{2}x_s + \frac{5}{6}x_d &\leq 600 && \text{(Sewing)} \\
x_s + \frac{2}{3}x_d &\leq 708 && \text{(Finishing)} \\
0.1x_s + 0.25x_d &\leq 135 && \text{(Inspection \& Packaging)} \\
x_s, x_d &\geq 0
\end{aligned}$$

Writing the PuLP Code

Gemini Prompt

Here is an linear program: max $Z = 10x_s + 9x_d$ s.t. $\frac{7}{10}x_s + x_d \leq 630; \frac{1}{2}x_s + \frac{5}{6}x_d; x_s + \frac{2}{3}x_d \leq 708; 0.1x_s + 0.25x_d \leq 135; x_s, x_d \geq 0$. Write me a python code that solves the linear program using PuLP. Also add code that will compute **reduced costs**, **shadow prices**, and **indicate which constraints are loose or tight**.

PuLP Code with Shadow Prices and Reduced Costs

```
import pulp as pl

# Model
prob = pl.LpProblem("Par_Inc_Golf_Bags", pl.LpMaximize)

# Variables (continuous  0; add cat='Integer' if you ever need integrality)
x_s = prob.add_variable("Standard_Bags", lowBound=0)
x_d = prob.add_variable("Deluxe_Bags",   lowBound=0)

# Objective
prob += 10 * x_s + 9 * x_d, "Maximize_Profit"

# Constraints
prob += 0.7 * x_s + 1.0   * x_d <= 630, "Cutting_Dyeing"
prob += 0.5 * x_s + (5/6) * x_d <= 600, "Sewing"
prob += 1.0 * x_s + (2/3) * x_d <= 708, "Finishing"
prob += 0.1 * x_s + 0.25  * x_d <= 135, "Inspection_Packaging"

# Solve with CBC so reduced costs are available
res = prob.solve()
```

PuLP Code with Shadow Prices and Reduced Costs

```
# Report primal solution
print("Status:", res.status.name)
print(f"Standard Bags   = {x_s.varValue:,.2f}")
print(f"Deluxe Bags     = {x_d.varValue:,.2f}")
print(f"Maximum Profit  = ${pl.value(prob.objective):,.2f}")

# Shadow Prices and Slacks
print("\nSensitivity Analysis -- Constraints:")
for c in prob.constraints():
    shadow_price = abs(c.pi) if c.pi is not None else 0.0
    slack = abs(c.slack) if c.slack is not None else 0.0
    print(f"  {c.name:<22}: Shadow Price = ${shadow_price:.4f}, 
        Slack = {slack:.2f} hrs")

# Reduced Costs
print("\nSensitivity Analysis -- Decision Variables:")
for v in prob.variables():
    print(f"  {v.name:<15}: Value = {v.varValue:,.2f}, Reduced Cost
        = ${v.dj:.4f}")
```

Optimal Solution

No need to read the code. Just run it to get this:

Optimal Production Plan

- **Standard Bags** (x_s): 540
- **Deluxe Bags** (x_d): 252

Maximum Profit

$$\$7,668.00$$

Binding Constraints

These resources are fully utilized (Slack $= 0$) and define the optimal solution.

- **Cutting/Dyeing** ($\mathbf{C_1}$): $0.7x_s + 1.0x_d \leq 630$
- **Finishing** ($\mathbf{C_3}$): $1.0x_s + \frac{2}{3}x_d \leq 708$

What Is an Extra Hour Worth? (Shadow Price)

Definition: Shadow Price (Dual Value)

The marginal change in optimal profit per 1-unit increase in resource capacity (RHS):

$$\text{Shadow Price } (\pi_i) = \frac{\Delta \text{Optimal Profit}}{\Delta \text{Resource Capacity } (b_i)}$$

Managerial Willingness to Pay

- **Upper Limit:** Maximum price/premium worth paying for extra bottleneck capacity.
- **Overtime Limit:** Paying up to **\$4.38/hr** in overtime for Cutting is profitable.

The Golden Rule of Slack

- If Slack $> \mathbf{0}$, then Shadow Price $= \$\mathbf{0.00}$.
- **Intuition:** Why pay for additional hours when surplus capacity is already idle?

Par, Inc. Shadow Prices: Where Is the Bottleneck?

Department	Available	Used	Slack	Shadow Price	Status
Cutting & Dyeing	630 hr	630 hr	0 hr	\$4.375 / hr	**Bottleneck**
Sewing	600 hr	480 hr	120 hr	\$0.000 / hr	Surplus
Finishing	708 hr	708 hr	0 hr	\$6.938 / hr	**Top Bottleneck**
Inspection & Pkg	135 hr	117 hr	18 hr	\$0.000 / hr	Surplus

Bottleneck Resources (Slack = 0)

- **Finishing (\$6.94/hr):** Most critical; highest priority for overtime/expansion.
- **Cutting & Dyeing (\$4.38/hr):** Second bottleneck; each extra hour yields \$4.38.

Surplus Resources (Slack > 0)

- **Sewing (120 idle hrs)** and **Inspection (18 idle hrs)**.
- Extra capacity generates **\$0.00** profit because resources exceed demand.

Reduced Cost: The Hurdle Rate for New Products

The Economic Hurdle Rate ("Pricing Out")

The **Reduced Cost** (d_j) measures how much more profitable an unproduced item must become before it is optimal to manufacture:

$$\text{Reduced Cost } (d_j) = \underbrace{\text{Unit Profit } (c_j)}_{\text{Revenue Earned}} - \underbrace{\sum_{i=1}^{m} a_{ij}\pi_i}_{\text{Opportunity Cost of Scarce Resources}}$$

Produced Items ($x_j^* > 0$)

- **Reduced Cost = \$0.00**
- Applies to Standard & Deluxe bags.
- Profit earned covers opportunity cost.

Unproduced Items ($x_j^* = 0$)

- **Reduced Cost** $\leq$ \$0.00
- Fails hurdle rate: consumes more value in capacity than it returns in profit.

The "Standard Plus" Bag: Pricing Out Resources

Marketing Proposal: New "Standard Plus" Model

Requires: 0.80 hr Cutting, 0.60 hr Sewing, 0.80 hr Finishing, and 0.15 hr Inspection.

Department	**Hours**	**Opp. Cost**
Cutting	0.80 hr	\$3.50
Sewing	0.60 hr	\$0.00
Finishing	0.80 hr	\$5.55
Inspection	0.15 hr	\$0.00
Hurdle Rate:		**\$9.05 / bag**

Surplus departments contribute \$0.00!

Key Economic Insight

- Hurdle rate: **\$9.05 / bag**.
- Diverting bottleneck hours **destroys \$9.05** profit.
- Viable only if unit profit $>$ \$9.05.

The Strategic Payoff: Fast Decisions Without Re-Solving

Hurdle Rate Decision Rule: d = Unit Profit − \$9.05

To be profitable, the unit profit must exceed the \$9.05 opportunity cost ($d > 0$).

Scenario A: Profit = \$10.00 / bag

$$d = \$10.00 - \$9.05 = +\$\mathbf{0.95} > \mathbf{0}$$

Decision: PRODUCE!

- Clears resource hurdle rate.
- Increases total company profit.

Scenario B: Profit = \$8.50 / bag

$$d = \$8.50 - \$9.05 = -\$\mathbf{0.55} < \mathbf{0}$$

Decision: REJECT!

- Fails resource hurdle rate.
- Destroys **\$0.55** profit per bag.

Executive Takeaway: Shadow prices allow managers to evaluate new initiatives in 15 seconds with simple mental arithmetic without re-solving complex LP models.

Case Study: Parket Sisters Pencil & Pen Factory

Multi-Product, Multi-Material Production Optimization

Products & Profit Margins

- X_1: **Ballpoint Pens** ($3.00 profit)
- X_2: **Mechanical Pencils** ($3.00 profit)
- X_3: **Fountain Pens** ($5.00 profit)

Material Inventory Limits

- **Plastic:** 1,000 oz available
- **Chrome:** 1,200 oz available
- **Stainless Steel:** 2,000 oz available

Linear Programming Model

$$\text{Maximize } Z = 3.0X_1 + 3.0X_2 + 5.0X_3$$

Subject to:

$$\begin{aligned} \text{Plastic:} \quad & 1.2X_1 + 1.7X_2 + 1.2X_3 \leq 1000 \\ \text{Chrome:} \quad & 0.8X_1 + 0.0X_2 + 2.3X_3 \leq 1200 \\ \text{Steel:} \quad & 2.0X_1 + 3.0X_2 + 4.5X_3 \leq 2000 \\ & X_1, X_2, X_3 \geq 0 \end{aligned}$$

Decisions Ahead

Extra steel offer, supply cut, and R&D redesign.

Parket Sisters: PuLP Optimization Dashboard

Baseline Primal Production Plan and Dual Resource Audit

Primal Production Plan (Optimal Objective: $Z^* = \$2,766.67$)

Product	Quantity (X_j^*)	Profit	Reduced Cost	Status
X_1 (Ballpoint)	700.00 units	\$3.00	\$0.0000	**In Production**
X_2 (Mech. Pencil)	0.00 units	\$3.00	-\$1.3833	**Excluded**
X_3 (Fountain Pen)	133.33 units	\$5.00	\$0.0000	**In Production**

Dual Resource Values & Capacity Utilization

Material	Available	Used	Slack	Shadow Price	Status
Plastic	1,000 oz	1,000 oz	0 oz	\$1.1667 / oz	**Bottleneck**
Chrome	1,200 oz	866.7 oz	333.3 oz	\$0.0000 / oz	Surplus
Stainless Steel	2,000 oz	2,000 oz	0 oz	\$0.8000 / oz	**Bottleneck**

Decision 1: Supplier Offer for Extra Steel

Capitalizing on Bottleneck Capacity Expansion

The Supplier's Offer

A distributor offers an additional **500 oz of Stainless Steel** at a **$0.60/oz premium** above standard contract price.

3-Step Evaluation

1. **Range of Feasibility:**
   - Allowable increase: 555.56 oz.
   - $500 \leq 555.56 \implies \pi_{\text{steel}} = \$\mathbf{0.80}$.
2. **Marginal Spread:**
   - Value: **$0.80/oz** vs. Cost: **$0.60/oz**
   - Net Spread: $+\$\mathbf{0.20}$/oz

3. Total Net Profit Impact

$$\Delta\text{Gross} = 500 \times \$0.80 = +\$400.00$$
$$\text{Cost} = 500 \times \$0.60 = -\$300.00$$
$$\mathbf{\Delta}\text{Net} = +\$\mathbf{100.00}$$

Managerial Decision

ACCEPT THE OFFER!

Net company profit increases by **+$100.00**.

Decision 2: Supplier Disruption & Chrome Shortage

Assessing Supply Chain Shocks Using Slack Buffers

Disruption Scenario

A logistics delay cuts Chrome supply from **1,200 oz down to 1,000 oz** (a 200 oz reduction). Should management panic?

Slack Buffer Analysis

- **Optimal Usage:** 866.67 oz
- **Available Slack:** **333.33** oz
- **Disruption:**
  200 oz $\leq$ 333.33 oz
- Non-binding
  $\implies$ $\pi_{\text{chrome}} = \$\mathbf{0.00}$

Impact on Operations

- **Profit Impact:** $\Delta Z^* = \$\mathbf{0.00}$
- **Production Mix:** $(700, 133.3)$ preserved
- **Remaining Slack:** **133.33** oz

Decision: DO NOT PANIC!

Buffer absorbs 100% of shock with zero loss.

Decision 3: R&D Redesign of Mechanical Pencil

Pricing Out a Product Redesign Using Shadow Prices

R&D Engineering Redesign Proposal

Original pencil is unprofitable ($d_2 = -\$1.38$). Redesigned pencil: profit remains **\$3.00**, but consumes **1.1 oz Plastic**, **2.0 oz Chrome**, and **2.0 oz Stainless Steel**.

Material	**Usage**	**Opp. Cost**
Plastic	1.1 oz	\$1.2833
Chrome (Surplus)	2.0 oz	\$0.0000
Stainless Steel	2.0 oz	\$1.6000
Total Hurdle Rate:		**\$2.8833**

Surplus Chrome adds \$0.00 to opportunity cost!

Hurdle Rate Evaluation

$$d_2^{\text{new}} = \$3.00 - \$2.88 = +\mathbf{\$0.12} > \mathbf{0}$$

Managerial Decision

APPROVE REDESIGN FOR PRODUCTION!

Clears hurdle rate; increases company profit.

Executive Summary: Sensitivity Decision Matrix

A 4-Quadrant Framework for Operational Decision-Making

1. Expanding Bottlenecks ($\pi_i > 0$)

- **Action:** Authorize overtime or supplier buy.
- **Pricing Rule:** Max price = Shadow Price (π_i).
- **Limit:** Valid up to Allowable Increase.

2. Surplus Resources ($\text{Slack}_i > 0$)

- **Action:** Never pay premiums for idle assets.
- **Pricing Rule:** Shadow price is strictly **\$0.00**.
- **Disruptions:** Absorbed free within slack.

3. New Products (Reduced Cost)

- **Action:** Price out scarce resource usage.
- **Hurdle Rule:** Launch if Unit Profit > Opp. Cost.
- **Costing:** Non-binding inputs cost **\$0.00**.

4. Price Changes (c_j)

- **Action:** Track market price fluctuations.
- **Rule:** Mix stays invariant within Range.
- **Profit:** Linear change $\Delta Z^* = x_j^* \Delta c_j$.